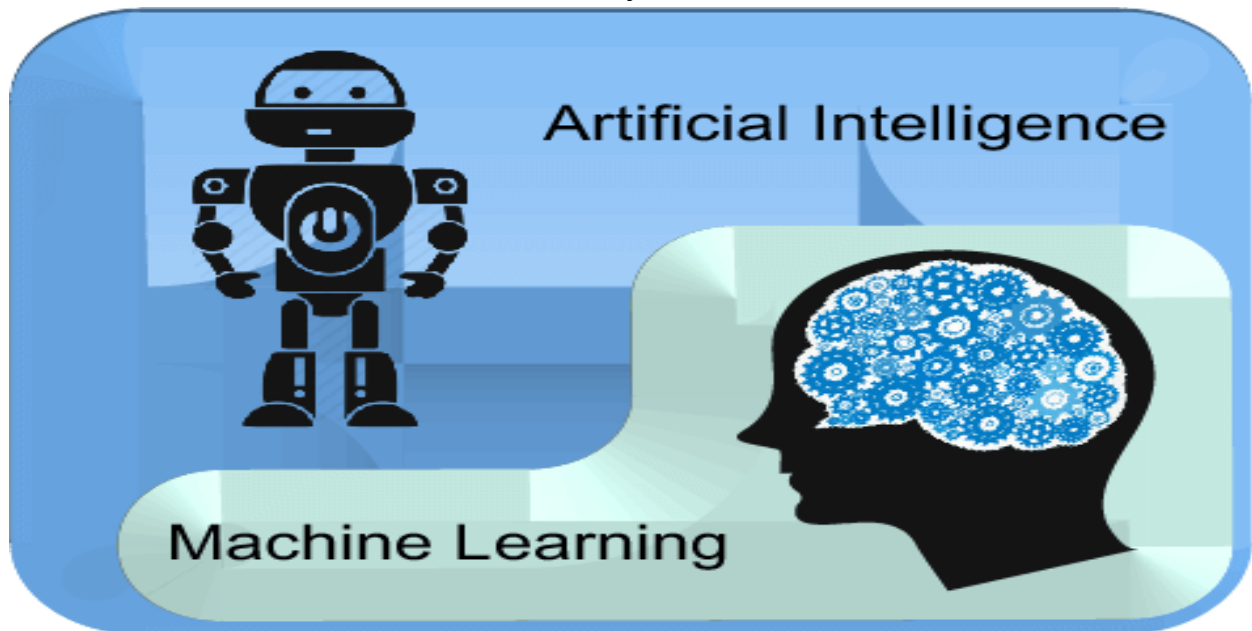


DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
Artificial Intelligence and Machine Learning



Machine Learning
24AM3501 (2024-2028)
Laboratory Manual



Dayananda Sagar University
Devarakaggalahalli, Harohalli Kanakpura Road , Bengaluru South, Karnataka – 562112

School of Engineering

Vision and Mission

Vision

To be a center of excellence in education, research & training, innovation & entrepreneurship, and to produce citizens with exceptional leadership qualities to serve national and global needs.

Mission

To achieve our objectives in an environment that enhances creativity, innovation and scholarly pursuits while adhering to our vision.

Values

The Pursuit of Excellence

A commitment to strive continuously to improve ourselves and our systems with the aim of becoming the best in our field.

Fairness

A commitment to objectivity and impartiality, to earn the trust and respect of society.

Leadership

A commitment to lead responsively and creatively in educational and research processes.

Integrity and Transparency

A commitment to be ethical, sincere and transparent in all activities and to treat all individuals with dignity and respect.

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Vision

To produce graduates in Computer Science and Engineering (Artificial Intelligence & Machine Learning) through excellence in education and research with an emphasis on sustainable ecosystem that contributes significantly to the society.

Mission

The Department Computer Science and Engineering (Artificial Intelligence & Machine Learning) is committed to:

- Impart quality education through the state-of-the-art curriculum, infrastructure facilities, cutting edge technologies, sustainable learning practices and lifelong learning.
- Collaborate with industry-academia and inculcate interdisciplinary research to transform professionals into technically competent.
- Produce engineers and techno-entrepreneurs for global needs.

Values

The values that drive DSU and support its vision:

The Pursuit of Excellence

A commitment to strive continuously to improve ourselves and our systems with the aim of becoming the best in our field.

Fairness

A commitment to objectivity and impartiality, to earn the trust and respect of society.

Leadership

A commitment to lead responsively and creatively in educational and research processes.

Integrity and Transparency

A commitment to be ethical, sincere and transparent in all activities and to treat all individuals with dignity and respect.



DAYANANDA SAGAR
UNIVERSITY



Program Educational Objectives (PEO's)

PEO1: Apply appropriate theory, practices, and tools of machine intelligence to the specification, design, implementation, maintenance, and evaluation in the workplace or in higher education.

PEO2: Adapt, contribute and innovate new technologies in their computing profession by working in teams to design, implement, and maintain in the key domains of Artificial Intelligence & Machine Learning.

PEO3: Function effectively in the work place as competent Artificial Intelligence & Machine Learning Professionals, Entrepreneurs or Researchers or maintain employment through lifelong learning such as professional conferences, certificate programs or other professional educational activities, ethics, and societal awareness.

Programme Outcome (PO's)

- ☐ **PO1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- ☐ **PO2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- ☐ **PO3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- ☐ **PO4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- ☐ **PO5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- ☐ **PO6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.



**DAYANANDA SAGAR
UNIVERSITY**



- ☐ **PO7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- ☐ **PO8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- ☐ **PO9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- ☐ **PO10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- ☐ **PO11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- ☐ **PO12. Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSO's)

PSO1: Cognitive Outcome: Solve complex engineering problems in computing by applying the principles in Artificial Intelligence, Machine Learning, Network Engineering, Software Engineering, Data Engineering and Intelligent Systems.

PSO2: Skill & Design Outcome: Apply technical skills and research skills through professional societies, certification programs, projects, internships and laboratory exercises to design & develop algorithms, programs, and projects using modern software tools to provide the sustainable solutions to Computer Science, Artificial Intelligence and Machine Learning problems related to the society and environment.

COURSE DETAILS			
#	Sem / Year	Course Code	Title of the Course
1	V	24AM3501	MACHINE LEARNING LABORATORY

Course Learning Objectives:

1. Summarize the basic concepts and different types of Machine Learning including Supervised learning, Unsupervised and Reinforcement learning Techniques.
2. Explore and analyze the mathematics behind Machine Learning algorithms to gain a solid understanding of probability density functions, basics of sampling theorem and estimation of the maximum likelihood.
3. Make Use of the Supervised Machine Learning techniques for solving appropriate real-world applications.
4. Make Use of the Unsupervised Machine Learning techniques, Feature Engineering and Dimensionality Reduction techniques for solving appropriate real-world applications.
5. Evaluate the performance of Machine Learning algorithms using appropriate metrics such as accuracy, precision, recall, FI Score and Make Use of the different Optimization and Regularization Techniques for improving the model performance.

COURSE OUTCOMES:

CO No.	Outcomes	Bloom's Taxonomy Level
1	Describe the basic concepts and different types of Machine Learning techniques including Supervised learning, Unsupervised learning and Reinforcement learning techniques and design Well Posed Learning system for solving the problem.	L6
2	Explore and Analyze the mathematics behind Machine Learning algorithms to gain a solid understanding of probability density functions, the basics of sampling theorem and estimating the maximum likelihood.	L6
3	Apply the learned concepts of machine learning to interpret the Supervised learning algorithms including regression and classification problems.	L4
4	Apply Unsupervised Machine Learning Algorithms such as Hard and Soft clustering and Feature Engineering and Dimensionality Reduction techniques for solving appropriate real-world applications.	L6
5	Evaluate the performance of Machine Learning algorithms using appropriate metrics such as accuracy, precision, recall, FI Score and Make Use of the different Optimization and Regularization Techniques for improving the model performance.	L6

Mapping Levels of COs to POs / PSOs														
COs				Program Outcomes (POs)									PSOs	
	1	2	3	4	5	6	7	8	9	10	11	12	1	2
	Engineering Knowledge	Problem Analysis	Design & Development	Conduct Investigations	Modern Tool Usage	The Engineer and Society	Environment and	Ethics	Individual & Team Work	Communication	Project management	Life-long Learning	Cognitive Outcome	Skill & Design
CO1	2	2	1	-	-	-	-	-	-	-	1	-	2	2
CO2	2	2	1	-	-	-	-	-	-	-	1	-	2	2
CO3	3	2	1	1	1	2	1	-	2	1	1	-	2	2
CO4	3	2	1	1	1	2	1	-	2	1	1	-	2	2
CO5	3	2	1	1	1	2	1	-	2	1	1	-	2	2

DAYANANDA SAGAR UNIVERSITY
SCHOOL OF ENGINEERING
Devarakaggalahalli, Harohalli Kanakpura Road , Bengaluru South, Karnataka – 562112

Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

List of Experiments

Exp No	Experiment Name
1	Perform Exploratory Data Analysis (EDA) on the given dataset
2	Apply Linear Regression on a given dataset and comment on their efficiency and performance. •Implement linear regression using sklearn library. •Split the obtained dataset into training and testing: 80-20, 70-30 ratio. •Evaluate the model using metrics: Accuracy, Mean Square Error.
3	Apply logistic regression to the given dataset and evaluate the model using a confusion matrix. Download the dataset from the repository and import it as input. Build a Logistic Regression Model and train the dataset. Evaluate the model performance using different parameters: Accuracy, Confusion Matrix, Precision, Recall, F score, and AUC-ROC curve.
4	Apply Naïve Bayes Classifier to the given dataset and evaluate the model using a confusion matrix Precision, Recall, F score, and AUC-ROC curve.
5	Use K Nearest Neighbor technique on a given dataset and analyze the performance by changing the value of K. •Implement the KNN algorithm. •Apply KNN model on the dataset and perform testing on unseen dataset. •Change the value of K in KNN and analysis the performance of the model.
6	To build a machine learning model using the Decision Tree algorithm to predict whether a breast tumor is benign or malignant based on various characteristics of the tumor, improving diagnostic accuracy in medical applications.
7	To apply a Random Forest Classifier for classifying breast cancer tumors as benign or malignant based on features like radius and texture.
8	Use Support Vector Machine to perform the classification and regression on given dataset. •Import the dataset and perform the classification using SVM. •Evaluate the model's performance on testing dataset and validation dataset •Use Sklearn's Grid Search CV method for fine tuning of hyper-parameters.
9	Apply K-Means Clustering on the collected dataset. •Implement K-Means clustering using sklearn. •Check for the best k-value.
10	Implement Principal Component Analysis (PCA) for dimensionality reduction and data visualization, demonstrating both from-scratch implementation and sklearn comparison on iris and digits datasets.
11	Implement both simple Perceptron and Multi-Layer Perceptron (MLP) neural networks, demonstrating

	binary classification with custom implementation and multi-class classification using TensorFlow/Keras.
12	Implement Byte Pair Encoding (BPE) for text processing, a tokenization technique that reduces vocabulary size by iteratively merging the most frequent character pairs in a corpus while retaining important sub word components.

EXPERIMENT 1

AIM: To perform Exploratory Data Analysis (EDA) on the given dataset using Pandas, Matplotlib, and Seaborn.

```
# =====
# EXPERIMENT 1: EXPLORATORY DATA ANALYSIS (EDA)
# =====

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#-----
# Load Dataset
#-----
df = pd.read_csv('/content/ai_workforce_displacement_global_2020_2026.csv')

sns.set(style="whitegrid")
plt.rcParams["figure.figsize"]=(8,5)

#-----
# Basic Information
#-----
print("="*60)
print("FIRST 5 RECORDS")
print(df.head())

print("\nSHAPE :",df.shape)

print("\nDATA TYPES")
print(df.dtypes)

print("\nMISSING VALUES")
print(df.isnull().sum())

print("\nDUPLICATES :",df.duplicated().sum())

print("\nSUMMARY STATISTICS")
print(df.describe())

print("\nCATEGORICAL FEATURES")
print(df.describe(include='object'))

# Numerical Columns
num_cols=df.select_dtypes(include='number').columns

#=====
# 1. Histogram
#=====
df[num_cols].hist(figsize=(16,12),bins=20)
plt.suptitle("1. Histogram of Numerical Features")
plt.tight_layout()
plt.show()

#=====
# 2. Distribution Plot
#=====
plt.figure()
sns.histplot(df['ai_adoption_index'],kde=True,color='steelblue')
plt.title("2. Distribution of AI Adoption Index")
plt.show()
```

```

=====
# 3. Box Plot
=====
plt.figure(figsize=(15,6))
sns.boxplot(data=df[num_cols])
plt.xticks(rotation=90)
plt.title("3. Box Plot")
plt.show()

=====
# 4. Correlation Heatmap
=====
plt.figure(figsize=(12,8))
sns.heatmap(df[num_cols].corr(),
            annot=True,
            cmap='coolwarm',
            fmt=".2f")
plt.title("4. Correlation Heatmap")
plt.show()

=====
# 5. Scatter Plot
=====
plt.figure()
sns.scatterplot(data=df,
                x='ai_adoption_index',
                y='pct_sector_workforce_displaced')
plt.title("5. AI Adoption vs Workforce Displacement")
plt.show()

=====
# 6. Scatter Plot with Hue
=====
plt.figure(figsize=(9,6))
sns.scatterplot(data=df,
                x='ai_adoption_index',
                y='pct_sector_workforce_displaced',
                hue='industry_sector')
plt.title("6. Scatter Plot with Industry")
plt.show()

=====
# 7. Regression Plot
=====
plt.figure()
sns.regplot(data=df,
            x='sector_automation_risk_score',
            y='pct_sector_workforce_displaced',
            scatter_kws={'alpha':0.6})
plt.title("7. Regression Plot")
plt.show()

=====
# 8. Count Plot
=====
plt.figure(figsize=(10,5))
sns.countplot(data=df,
              y='industry_sector',
              order=df['industry_sector'].value_counts().index)
plt.title("8. Industry Count")
plt.show()

=====

```

9. Bar Plot

```
=====
plt.figure(figsize=(10,5))
sns.barplot(data=df,
            x='industry_sector',
            y='ai_adoption_index',
            estimator='mean')
plt.xticks(rotation=45)
plt.title("9. Average AI Adoption by Industry")
plt.show()
```

10. Pie Chart

```
=====
plt.figure(figsize=(7,7))
df['industry_sector'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%'
)
plt.ylabel("")
plt.title("10. Industry Distribution")
plt.show()
```

11. Line Plot

```
=====
plt.figure()
sns.lineplot(data=df,
            x='year',
            y='ai_adoption_index',
            estimator='mean',
            errorbar=None)
plt.title("11. AI Adoption Over Years")
plt.show()
```

12. Violin Plot

```
=====
plt.figure(figsize=(10,5))
sns.violinplot(data=df,
            x='industry_sector',
            y='pct_sector_workforce_displaced')
plt.xticks(rotation=45)
plt.title("12. Violin Plot")
plt.show()
```

13. Strip Plot

```
=====
plt.figure(figsize=(10,5))
sns.stripplot(data=df,
            x='industry_sector',
            y='pct_sector_workforce_displaced',
            jitter=True)
plt.xticks(rotation=45)
plt.title("13. Strip Plot")
plt.show()
```

14. Pair Plot

```
=====
pair_cols=[
'sector_automation_risk_score',
'ai_adoption_index',
```

```
'pct_sector_workforce_displaced',
'pct_sector_workforce_new_roles_created',
'ai_tool_adoption_pct'
]
```

```
sns.pairplot(df[pair_cols])
plt.title("14. Pair Plot")
plt.show()
```

```
print("\nEDA COMPLETED SUCCESSFULLY")
```

Output:

```
=====
FIRST 5 RECORDS
```

	record_id	country	iso3_code	region	income_group	year	\
0	1	United States	USA	North America	High Income	2020	
1	2	United States	USA	North America	High Income	2020	
2	3	United States	USA	North America	High Income	2020	
3	4	United States	USA	North America	High Income	2020	
4	5	United States	USA	North America	High Income	2020	

	quarter	quarter_label	industry_sector	\
0	1	2020-Q1	Technology & Software	
1	1	2020-Q1	Finance & Banking	
2	1	2020-Q1	Healthcare & Life Sciences	
3	1	2020-Q1	Manufacturing & Industry	
4	1	2020-Q1	Retail & E-Commerce	

	sector_automation_risk_score	...	pct_sector_workforce_new_roles_created	\
0	0.382	...	0.0348	
1	0.608	...	0.0423	
2	0.198	...	0.0168	
3	0.720	...	0.0642	
4	0.676	...	0.0374	

	net_workforce_change_pct	ai_cited_layoff_announcements	\
0	-0.0058	32	
1	-0.0094	51	
2	-0.0008	4	
3	-0.0282	47	
4	-0.0293	31	

	ai_skill_wage_premium_pct	pct_workforce_female	\
0	0.311	0.28	
1	0.304	0.48	
2	0.278	0.72	
3	0.268	0.29	
4	0.313	0.54	

	pct_displaced_roles_female	reskilling_programs_count	\
0	0.275	52	
1	0.511	39	
2	0.707	37	
3	0.310	50	
4	0.547	37	

	govt_ai_policy_score_1_to_10	ai_tool_adoption_pct	\
0	6.4	0.280	
1	6.5	0.559	
2	6.8	0.202	
3	6.8	0.655	
4	6.2	0.542	

```

                                data_source_notes
0  Research-calibrated synthetic data. Grounded i...
1  Research-calibrated synthetic data. Grounded i...
2  Research-calibrated synthetic data. Grounded i...
3  Research-calibrated synthetic data. Grounded i...
4  Research-calibrated synthetic data. Grounded i...

```

[5 rows x 23 columns]

SHAPE : (20800, 23)

```

DATA TYPES
record_id          int64
country            object
iso3_code          object
region            object
income_group       object
year              int64
quarter            int64
quarter_label      object
industry_sector    object
sector_automation_risk_score float64
gdp_per_capita_usd int64
ai_adoption_index  float64
pct_sector_workforce_displaced float64
pct_sector_workforce_new_roles_created float64
net_workforce_change_pct float64
ai_cited_layoff_announcements int64
ai_skill_wage_premium_pct float64
pct_workforce_female float64
pct_displaced_roles_female float64
reskilling_programs_count int64
govt_ai_policy_score_1_to_10 float64
ai_tool_adoption_pct float64
data_source_notes  object
dtype: object

```

```

MISSING VALUES
record_id          0
country            0
iso3_code          0
region            0
income_group       0
year              0
quarter            0
quarter_label      0
industry_sector    0
sector_automation_risk_score 0
gdp_per_capita_usd 0
ai_adoption_index  0
pct_sector_workforce_displaced 0
pct_sector_workforce_new_roles_created 0
net_workforce_change_pct 0
ai_cited_layoff_announcements 0
ai_skill_wage_premium_pct 0
pct_workforce_female 0
pct_displaced_roles_female 0
reskilling_programs_count 0
govt_ai_policy_score_1_to_10 0
ai_tool_adoption_pct 0
data_source_notes  0
dtype: int64

```

DUPLICATES : 0

SUMMARY STATISTICS

	record_id	year	quarter	sector_automation_risk_score \
count	20800.000000	20800.000000	20800.000000	20800.000000
mean	10400.500000	2022.769231	2.423077	0.537881
std	6004.587135	1.887406	1.115411	0.190345
min	1.000000	2020.000000	1.000000	0.151000
25%	5200.750000	2021.000000	1.000000	0.379000
50%	10400.500000	2023.000000	2.000000	0.580000
75%	15600.250000	2024.000000	3.000000	0.698000
max	20800.000000	2026.000000	4.000000	0.855000

	gdp_per_capita_usd	ai_adoption_index	pct_sector_workforce_displaced \
count	20800.000000	20800.000000	20800.000000
mean	21003.330769	0.692148	0.056155
std	22048.862955	0.161194	0.044811
min	879.000000	0.169000	0.001800
25%	3456.500000	0.591000	0.023300
50%	10635.000000	0.720000	0.043400
75%	40528.500000	0.821000	0.074900
max	99074.000000	0.963000	0.300000

	pct_sector_workforce_new_roles_created	net_workforce_change_pct \
count	20800.000000	20800.000000
mean	0.035631	-0.020524
std	0.036586	0.015327
min	0.000300	-0.140300
25%	0.009800	-0.028200
50%	0.021900	-0.016400
75%	0.049400	-0.009100
max	0.279100	-0.000300

	ai_cited_layoff_announcements	ai_skill_wage_premium_pct \
count	20800.000000	20800.000000
mean	88.346538	0.361570
std	85.198076	0.094132
min	0.000000	0.063000
25%	24.000000	0.293000
50%	63.000000	0.366000
75%	126.000000	0.435000
max	603.000000	0.610000

	pct_workforce_female	pct_displaced_roles_female \
count	20800.000000	20800.000000
mean	0.462000	0.497154
std	0.188833	0.222239
min	0.220000	0.213000
25%	0.280000	0.290000
50%	0.470000	0.493000
75%	0.580000	0.610000
max	0.790000	0.950000

	reskilling_programs_count	govt_ai_policy_score_1_to_10 \
count	20800.000000	20800.000000
mean	34.497981	5.370067
std	22.688847	1.566616
min	2.000000	1.100000
25%	13.000000	4.000000
50%	30.000000	5.400000
75%	53.000000	6.800000
max	92.000000	8.800000

	ai_tool_adoption_pct
count	20800.000000
mean	0.372712

```
std          0.166875
min          0.037000
25%          0.231000
50%          0.365000
75%          0.498000
max          0.881000
```

CATEGORICAL FEATURES

```
count      country iso3_code  region income_group quarter_label \
unique           80         80      12         4         26
top      United States      USA  Europe  High Income      2020-Q1
freq           260        260    5200        8060        800
```

```
count      industry_sector \
unique           10
top      Technology & Software
freq           2080
```

```
count      data_source_notes
unique           1
top      Research-calibrated synthetic data. Grounded i...
freq           20800
```

Note: Analyze the all visuals (Plots) when it is executed.

EDA COMPLETED SUCCESSFULLY

EXPERIMENT 2

AIM: Apply Linear Regression on a given dataset and comment on their efficiency and performance.

Objectives:

- Implement linear regression using sklearn library
- Split the obtained dataset into training and testing: 80-20, 70-30 ratio
- Evaluate the model using metrics: Accuracy, Mean Square Error

Theory:

Linear regression models the relationship between a dependent variable (Y) and an independent variable (X) using the equation $Y = mX + c$. The slope m and intercept c are determined by minimizing the error, calculated using the Quadratic Loss Function. The Least Squares method provides the optimal values for m and c by minimizing the total squared difference between actual and predicted values.

```
import pandas as pd
import numpy as np

# Generate synthetic data with linear relationship np.random.seed(42)
X = np.arange(1, 21)
Y = 2 * X + 1 + np.random.normal(0, 2, 20) # Y = 2X + 1 + noise

# Create DataFrame and save
data = pd.DataFrame({'X': X, 'Y': Y})
data.to_csv('data.csv', index=False) print("Dataset
created successfully!") print(data.head())
```

Dataset created successfully!

	X	Y
0	1	3.993428
1	2	4.723471
2	3	8.295377
3	4	12.046060
4	5	10.531693

$$m = \frac{\sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2} \dots\dots\dots (i),$$

and

$$c = \bar{y} - m\bar{x} \dots\dots\dots (ii)$$

```
# Making imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
plt.rcParams['figure.figsize'] = (12.0, 9.0)
```

```

# Preprocessing Input data
data = pd.read_csv('data.csv')
X = data.iloc[:, 0]
Y = data.iloc[:, 1]
plt.scatter(X, Y)
plt.title('Original Dataset') plt.show()

# Manual Linear Regression Function
def manual_linear_regression(X_train, Y_train): X_mean =
    np.mean(X_train)
    Y_mean = np.mean(Y_train)
    num = 0
    den = 0
    for i in range(len(X_train)):
        num += (X_train.iloc[i] - X_mean) * (Y_train.iloc[i] - Y_mean) den += (X_train.iloc[i]
            - X_mean) ** 2

    m = num / den
    c = Y_mean - m * X_mean
    return m, c
# Split dataset - 80:20 ratio
X_train_80, X_test_20, Y_train_80, Y_test_20 = train_test_split(X, Y, test_size=0.2, random_state=42)

# Split dataset - 70:30 ratio
X_train_70, X_test_30, Y_train_70, Y_test_30 = train_test_split(X, Y, test_size=0.3, random_state=42)

# Model 1: 80-20 Split
m_80, c_80 = manual_linear_regression(X_train_80, Y_train_80)
Y_pred_80 = m_80 * X_test_20 + c_80

# Model 2: 70-30 Split
m_70, c_70 = manual_linear_regression(X_train_70, Y_train_70)
Y_pred_70 = m_70 * X_test_30 + c_70

# Calculate metrics for 80-20 split
MSE_80 = mean_squared_error(Y_test_20, Y_pred_80) accuracy_80 =
r2_score(Y_test_20, Y_pred_80)

print("80-20 Split Results:")
print(f"Slope (m): {m_80:.4f}")
print(f"Intercept (c): {c_80:.4f}")
print(f"MSE: {MSE_80:.4f}")
print(f"Accuracy (R2 Score): {accuracy_80:.4f}")

# Calculate metrics for 70-30 split
MSE_70 = mean_squared_error(Y_test_30, Y_pred_70) accuracy_70 =
r2_score(Y_test_30, Y_pred_70)

print("\n70-30 Split Results:")
print(f"Slope (m): {m_70:.4f}")
print(f"Intercept (c): {c_70:.4f}")
print(f"MSE: {MSE_70:.4f}")
print(f"Accuracy (R2 Score): {accuracy_70:.4f}")

# Visualization
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X_test_20, Y_test_20, alpha=0.7, label='Actual')
plt.plot([min(X_test_20), max(X_test_20)], [min(Y_pred_80), max(Y_pred_80)], color='red', label='Predicted') plt.title('80-20 Split')
plt.legend()

plt.subplot(1, 2, 2)

```

```
plt.scatter(X_test_30, Y_test_30, alpha=0.7, label='Actual')
plt.plot([min(X_test_30), max(X_test_30)], [min(Y_pred_70), max(Y_pred_70)], color='red', label='Predicted') plt.title('70-30 Split')
plt.legend() plt.show()

# Performance Analysis
print("\n" + "="*50)
print("PERFORMANCE ANALYSIS:")
print("="*50)

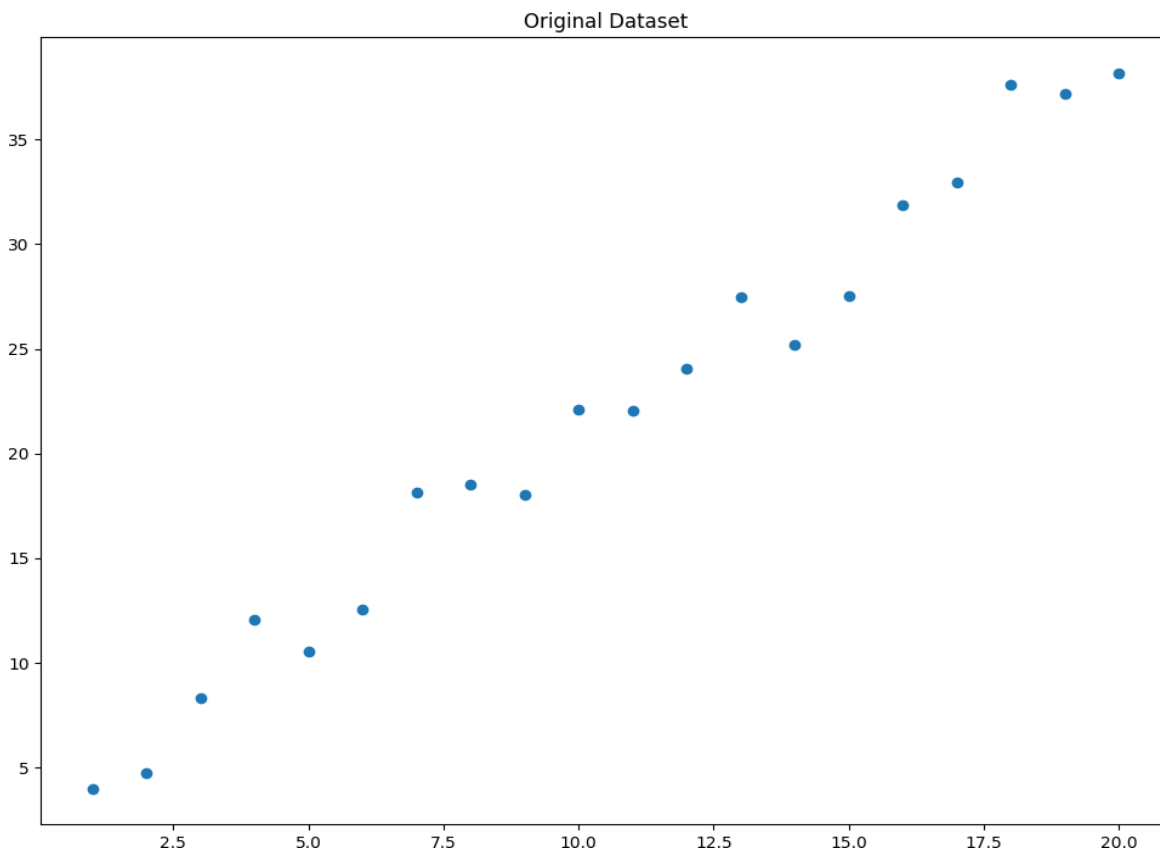
if MSE_80 < MSE_70:
    print("• 80-20 split performs BETTER (Lower MSE)") else:
    print("• 70-30 split performs BETTER (Lower MSE)")

if accuracy_80 > accuracy_70:
    print("• 80-20 split has HIGHER accuracy (R² Score)") else:
    print("• 70-30 split has HIGHER accuracy (R² Score)")

print(f"\nModel Efficiency:")
print(f"• 80-20 Split: MSE={MSE_80:.4f}, Accuracy={accuracy_80:.4f}") print(f"• 70-30 Split:
MSE={MSE_70:.4f}, Accuracy={accuracy_70:.4f}")

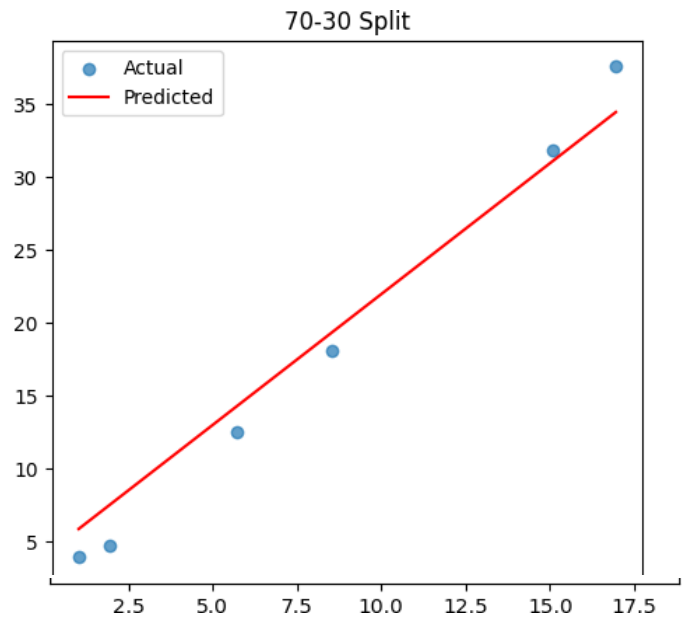
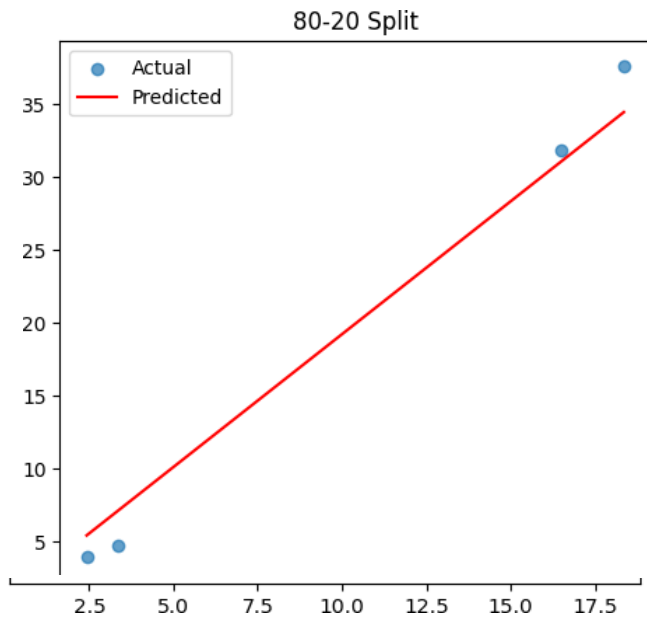
print(f"\nConclusion:")
if accuracy_80 > 0.8 and accuracy_70 > 0.8:
    print("• Both models show GOOD performance (R² > 0.8)") elif accuracy_80
> 0.6 or accuracy_70 > 0.6:
    print("• Models show MODERATE performance") else:
    print("• Models show POOR performance - consider feature engineering")

print("• Lower MSE indicates better fit")
print("• Higher R² Score indicates better accuracy")
```



80-20 Split Results:
Slope (m): 1.7065
Intercept (c): 3.7318
MSE: 4.6926
Accuracy (R² Score): 0.9800

70-30 Split Results:
Slope (m): 1.6806
Intercept (c): 4.2001
MSE: 4.4997
Accuracy (R² Score): 0.9725



=====

PERFORMANCE ANALYSIS:

=====

- 70-30 split performs BETTER (Lower MSE)
- 80-20 split has HIGHER accuracy (R² Score)

Model Efficiency:

- 80-20 Split: MSE=4.6926, Accuracy=0.9800
- 70-30 Split: MSE=4.4997, Accuracy=0.9725

Conclusion:

- Both models show GOOD performance (R² > 0.8)
- Lower MSE indicates better fit
- Higher R² Score indicates better accuracy

EXPERIMENT 3

AIM: Apply logistic regression to the given dataset and evaluate the model using a confusion matrix.

Objectives:

- Download the dataset from the repository and import it as input
- Build a Logistic Regression Model and train the dataset
- Evaluate the model performance using different parameters: Accuracy, Confusion Matrix, Precision, Recall, F score, and AUC-ROC curve

Theory:

Logistic Regression is a classification algorithm used to predict the probability of a binary outcome. It models the relationship between input features (X) and a binary target variable (Y) by estimating the probability of class membership using the sigmoid function. The model outputs probabilities that are thresholded to classify instances into one of two categories (0 or 1).

The performance is assessed using metrics like accuracy, confusion matrix, and ROC curve, which illustrate the model's ability to distinguish between positive and negative classes. Standardization ensures that the features are scaled for better model performance.

		Predicted class		
		Classified positive	Classified negative	
Actual class	Actual positive	TP	FN	TPR: $\frac{TP}{TP + FN}$
	Actual negative	FP	TN	FPR: $\frac{TN}{TN + FP}$

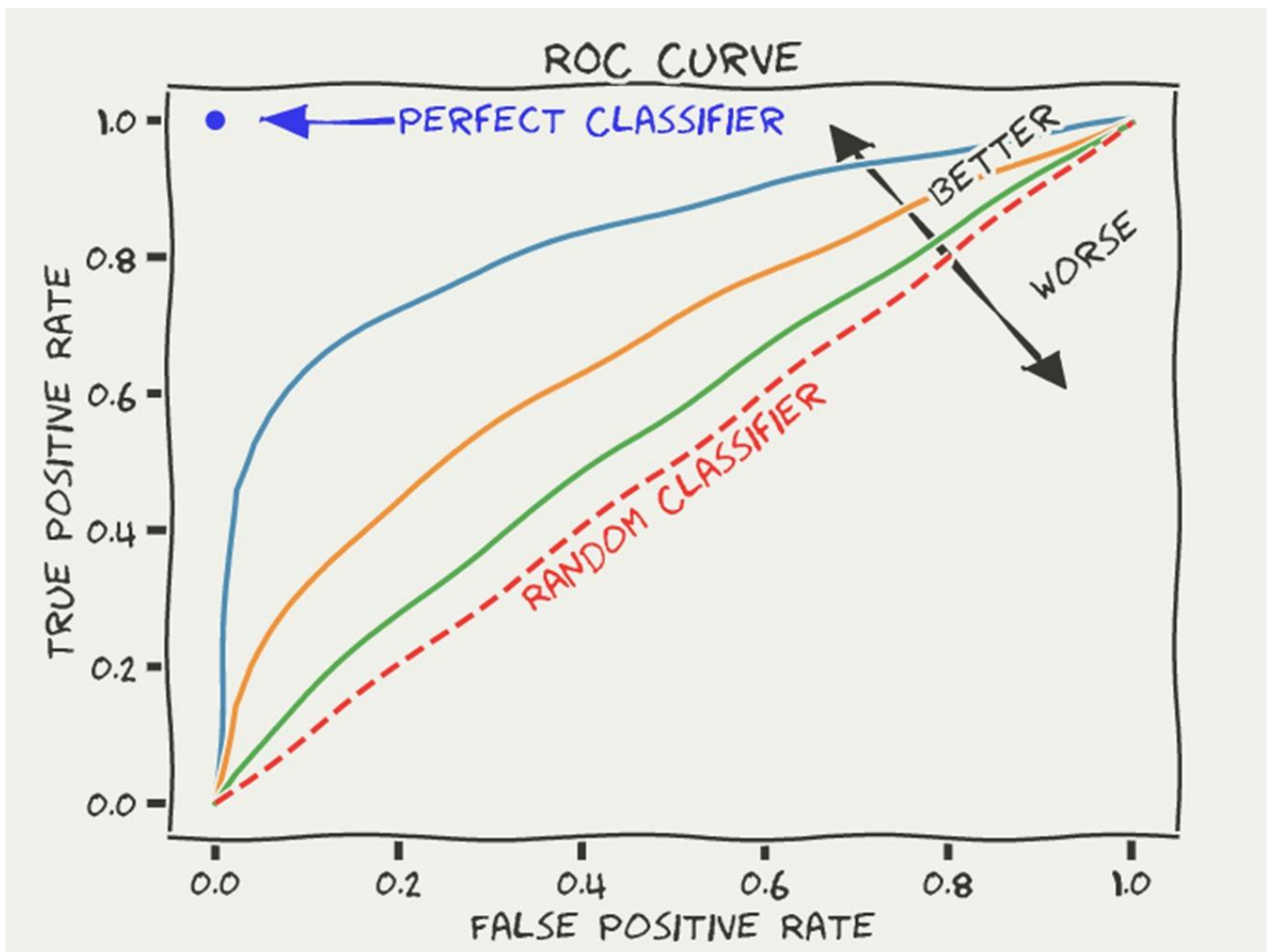
$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

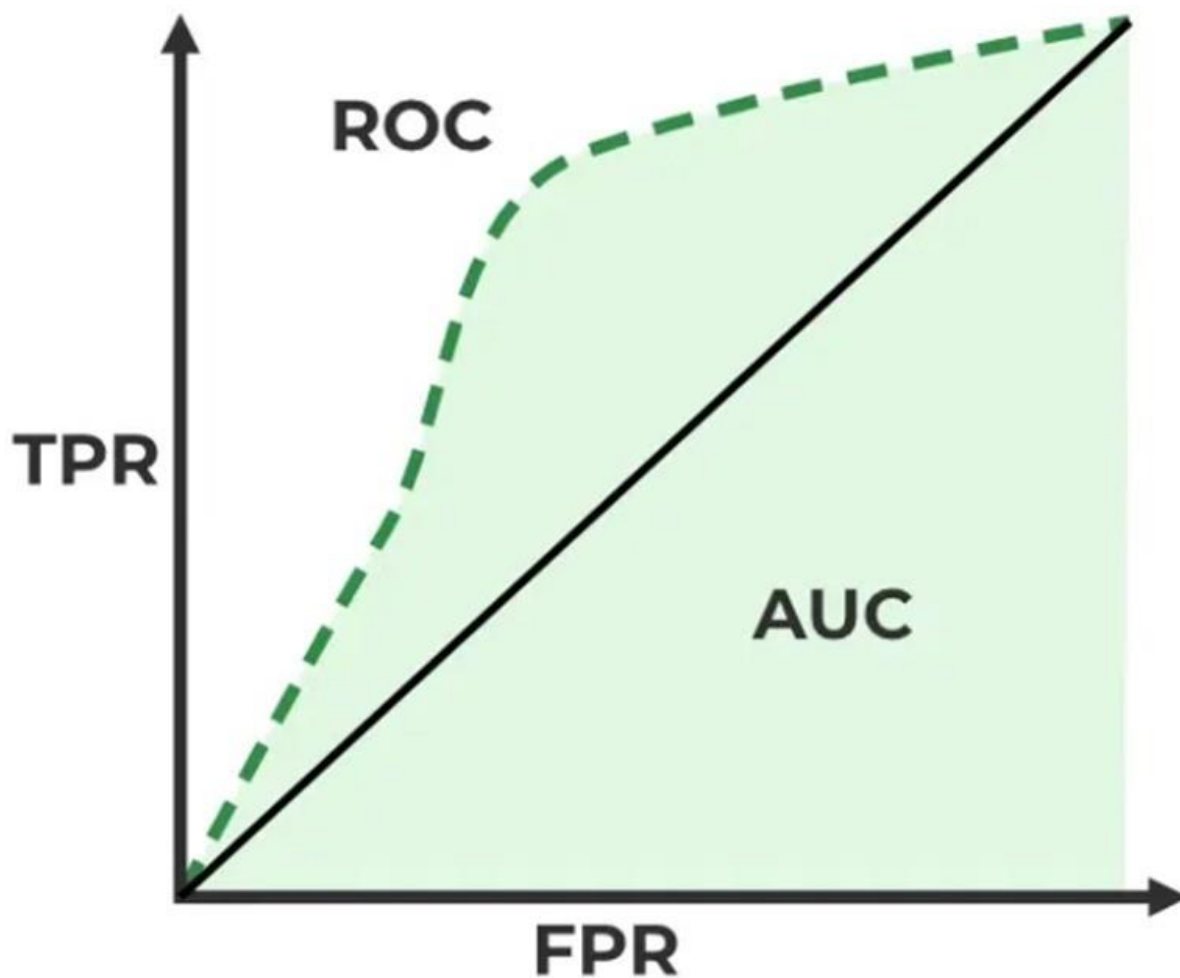
$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

$$\text{F1 score} = 2 \times \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$





EXPERIMENT: 3

AIM: Apply Logistic Regression on the given dataset and perform the model evaluation using confusion matrix # • Download the dataset from repository and import it as input

• Build Logistic Regression Model and train the dataset

• Evaluate the model performance using different parameters: Accuracy, Confusion Matrix, Precision, Recall, F score,

Theory: Logistic Regression is a classification algorithm used to predict the probability of a binary outcome. # It models the relationship between input features (X) and a binary target variable (Y) by estimating the probability of class membership using the sigmoid function.

Import necessary libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, auc
```

```
# Load the heart disease dataset
heart_data = pd.read_csv('heart.csv')
print("Dataset shape:", heart_data.shape)
print("\nFirst 5 rows:")
print(heart_data.head())
```

```

# Check for missing values
print("\nMissing values in each column:")
print(heart_data.isnull().sum())

# Handle missing values
print(f"\nDataset shape before handling missing values: {heart_data.shape}")

# Fill missing values with median for numerical columns
numerical_columns = heart_data.select_dtypes(include=[np.number]).columns
for col in numerical_columns:
    if heart_data[col].isnull().sum() > 0:
        heart_data[col].fillna(heart_data[col].median(), inplace=True)

# Alternative: Drop rows with missing values (uncomment if preferred)
# heart_data = heart_data.dropna()

print(f"Dataset shape after handling missing values: {heart_data.shape}")
print("Missing values after handling:")
print(heart_data.isnull().sum())

# Separate features and target variable
X = heart_data.drop('Ten YearCHD', axis=1) # Features
y = heart_data['Ten YearCHD'] # Target variable (0: no heart disease, 1: heart disease)

print(f"\nTarget distribution:\n{y.value_counts()}")

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train the Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"\nActual vs Predicted (first 10):")
print("Actual: ", y_test.values[:10])
print("Predicted: ", y_pred[:10])
print("Accuracy: {:.2f}%".format(accuracy * 100))

# Display Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Plot Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No Disease', 'Disease'], yticklabels=['No Disease', 'Disease'])
plt.title('Confusion Matrix\nAccuracy: {:.2f}%'.format(accuracy * 100))
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

```

# Visualize feature importance (coefficients)
feature_names = heart_data.drop('TenYearCHD', axis=1).columns

coefficients = model.coef_[0]

plt.figure(figsize=(10, 6))
feature_importance = pd.DataFrame({
    'feature': feature_names,
    'coefficient': coefficients
}).sort_values('coefficient', key=abs, ascending=False)

sns.barplot(data=feature_importance, x='coefficient', y='feature')
plt.title('Feature Importance (Logistic Regression Coefficients)')
plt.xlabel('Coefficient Value')
plt.show()

# Plot ROC Curve
y_prob = model.predict_proba(X_test)[: , 1]
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve\nAccuracy: {:.2f}%'.format(accuracy * 100))
plt.legend(loc="lower right")
plt.grid(True) plt.show()

# Additional Performance Metrics
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_test, y_pred) recall =
recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\n" + "="*50)
print("PERFORMANCE SUMMARY:")
print("="*50)
print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)") print(f"Precision:
{precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"AUC-ROC: {roc_auc:.4f}")

# Performance Analysis
print("\nModel Performance Analysis:") if accuracy >
0.85:
    print("• EXCELLENT model performance") elif
accuracy > 0.75:
    print("• GOOD model performance") elif
accuracy > 0.65:
    print("• MODERATE model performance") else:
    print("• POOR model performance - needs improvement")

if roc_auc > 0.9:
    print("• EXCELLENT discrimination ability") elif roc_auc
> 0.8:
    print("• GOOD discrimination ability") else:
    print("• MODERATE discrimination ability")

```

Dataset shape: (4238, 16)

First 5 rows:

	male	age	education	currentSmoker	cigsPerDay	BPMeds	prevalentStroke	\
0	1	39		4.0	0	0.0	0.0	0
1	0	46		2.0	0	0.0	0.0	0
2	1	48		1.0	1	20.0	0.0	0
3	0	61		3.0	1	30.0	0.0	0
4	0	46		3.0	1	23.0	0.0	0

	prevalentHyp	diabetes	totChol	sysBP	diaBP	BMI	heartRate	glucose	\
0		0	0	195.0	106.0	70.0	26.97	80.0	77.0
1		0	0	250.0	121.0	81.0	28.73	95.0	76.0
2		0	0	245.0	127.5	80.0	25.34	75.0	70.0
3		1	0	225.0	150.0	95.0	28.58	65.0	103.0
4		0	0	285.0	130.0	84.0	23.10	85.0	85.0

	TenYearCHD
0	0
1	0
2	0
3	1
4	0

Missing values in	each column:
male	0
age	0
education	105
currentSmoker	0
cigsPerDay	29
BPMeds	53
prevalentStroke	0
prevalentHyp	0
diabetes	0
totChol	50
sysBP	0
diaBP	0
BMI	19
heartRate	1
glucose	388
TenYearCHD	0
dtype: int64	

Dataset shape before handling missing values: (4238, 16)

Dataset shape after handling missing values: (4238, 16)

Missing values after handling:

male	0
age	0
education	0
currentSmoker	0
cigsPerDay	0
BPMeds	0
prevalentStroke	0
prevalentHyp	0
diabetes	0
totChol	0
sysBP	0
diaBP	0
BMI	0
heartRate	0
glucose	0
TenYearCHD	0
dtype: int64	

Target distribution:

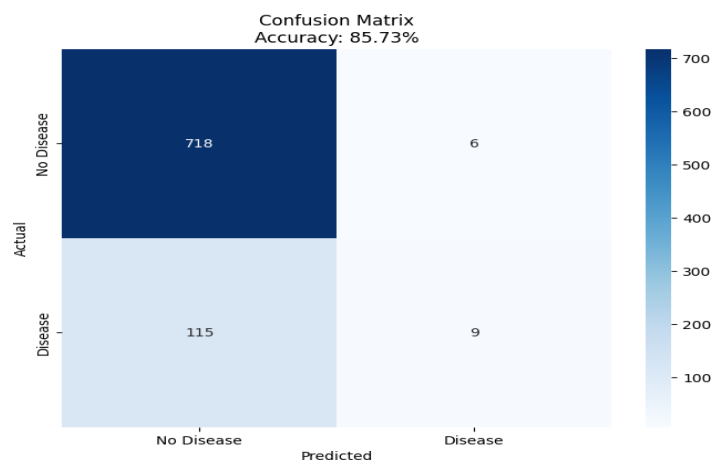
TenYearCHD	
0	3594
1	644

Name: count, dtype: int64

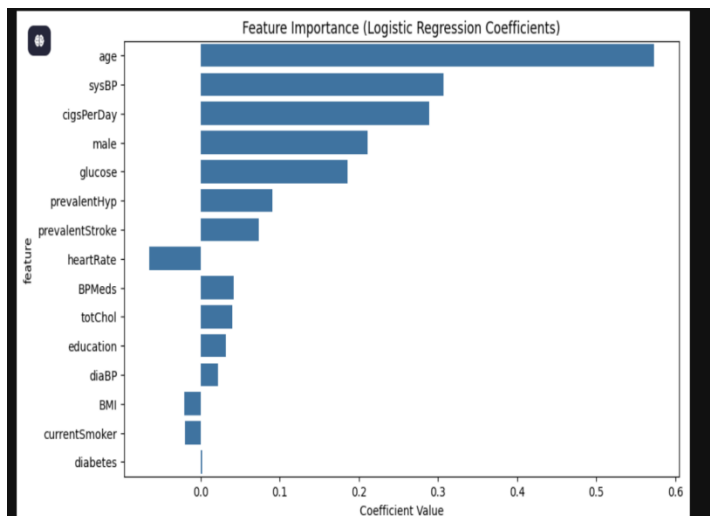
Actual vs Predicted (first 10):

Predicted: [0 0 0 0 0 0 0 0 0]
Accuracy: 85.73%

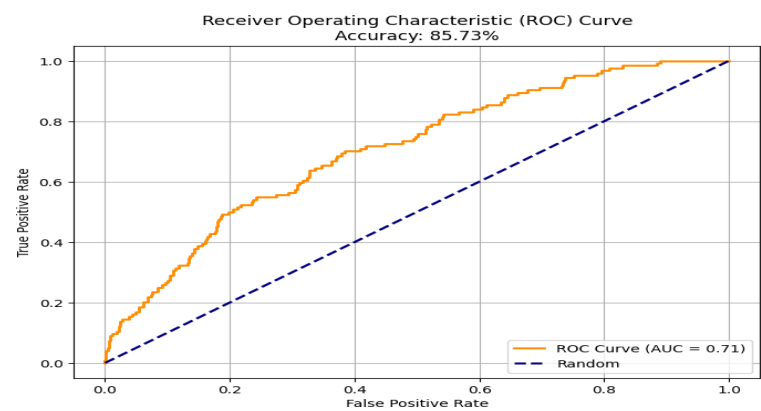
Confusion Matrix:
[[718 6]
[115 9]]



Classification Report:



Classification Report:



=====

PERFORMANCE SUMMARY:

=====

Accuracy: 0.8573 (85.73%)
Precision: 0.6000
Recall: 0.0726
F1-Score: 0.1295
AUC-ROC: 0.7072

- Model Performance Analysis:
- EXCELLENT model performance
 - MODERATE discrimination ability

EXPERIMENT 4

AIM: Apply Naïve Bayes Classifier to the given dataset and evaluate the model using a confusion matrix Precision, Recall, F score, and AUC-ROC curve.

Theory:

Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem, assuming independence between features. The Gaussian Naive Bayes variant is particularly useful when features follow a normal distribution. It calculates the probability of each class and classifies based on the highest posterior probability. The model is evaluated using metrics like accuracy, precision-recall, and ROC curves to assess its predictive performance.

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics
from sklearn.metrics import accuracy_score, classification_report, precision_recall_curve
from sklearn.metrics import confusion_matrix, f1_score, roc_curve, auc

# Read Titanic dataset
df = pd.read_csv('titanic.csv')
print("Dataset shape:", df.shape)
print("\nColumnnames:")
print(df.columns.tolist())
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset info:")
print(df.info())

# Data preprocessing - Select relevant features
df_processed = df[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked', 'Survived']].copy()
print("\nMissing values:")
print(df_processed.isnull().sum())

# Handle missing values
df_processed['Age'].fillna(df_processed['Age'].median(), inplace=True)
df_processed['Fare'].fillna(df_processed['Fare'].median(), inplace=True)
df_processed['Embarked'].fillna(df_processed['Embarked'].mode()[0], inplace=True)
print("\nMissing values after handling:")
print(df_processed.isnull().sum())

# Encode categorical variables
le_sex = LabelEncoder()
le_embarked = LabelEncoder()
df_processed['Sex'] = le_sex.fit_transform(df_processed['Sex'])
df_processed['Embarked'] = le_embarked.fit_transform(df_processed['Embarked'])
print("\nDataset after encoding:")
print(df_processed.head())

# Exploratory Data Analysis with proper formatting plt.style.use('default')
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Titanic Dataset - Exploratory Data Analysis', fontsize=16, fontweight='bold')

# Survival distribution
axes[0, 0].bar(['Not Survived', 'Survived'], df_processed['Survived'].value_counts(), color=['red', 'green'], alpha=0.7)
axes[0, 0].set_title('Survival Distribution', fontsize=12, fontweight='bold')
axes[0, 0].set_xlabel('SurvivalStatus', fontsize=10)
```

```

axes[0, 0].set_ylabel('Count', fontsize=10)

for i, v in enumerate(df_processed['Survived'].value_counts()):
    axes[0, 0].text(i, v + 10, str(v), ha='center', fontweight='bold')

# Age distribution
axes[0, 1].hist(df_processed['Age'], bins=30, alpha=0.7, color='skyblue', edgecolor='black')
axes[0, 1].set_title('AgeDistribution', fontsize=12, fontweight='bold')
axes[0, 1].set_xlabel('Age (years)', fontsize=10)
axes[0, 1].set_ylabel('Frequency', fontsize=10)
axes[0, 1].grid(axis='y', alpha=0.3)

# Fare distribution
axes[0, 2].hist(df_processed['Fare'], bins=30, alpha=0.7, color='orange', edgecolor='black')
axes[0, 2].set_title('Fare Distribution', fontsize=12, fontweight='bold')
axes[0, 2].set_xlabel('Fare ($)', fontsize=10)
axes[0, 2].set_ylabel('Frequency', fontsize=10)
axes[0, 2].grid(axis='y', alpha=0.3)

# Survival by Class
survival_by_class = pd.crosstab(df_processed['Pclass'], df_processed['Survived'])
survival_by_class.plot(kind='bar', ax=axes[1, 0], color=['red', 'green'], alpha=0.7)
axes[1, 0].set_title('Survival by Passenger Class', fontsize=12, fontweight='bold')
axes[1, 0].set_xlabel('Passenger Class', fontsize=10)
axes[1, 0].set_ylabel('Count', fontsize=10)
axes[1, 0].legend(['Not Survived', 'Survived'], loc='upper right')
axes[1, 0].set_xticklabels(['1st Class', '2nd Class', '3rd Class'], rotation=0)

# Survival by Sex
survival_by_sex = pd.crosstab(df_processed['Sex'], df_processed['Survived'])
survival_by_sex.plot(kind='bar', ax=axes[1, 1], color=['red', 'green'], alpha=0.7)
axes[1, 1].set_title('Survival by Gender', fontsize=12, fontweight='bold')
axes[1, 1].set_xlabel('Gender', fontsize=10)
axes[1, 1].set_ylabel('Count', fontsize=10)
axes[1, 1].legend(['Not Survived', 'Survived'], loc='upper right')
axes[1, 1].set_xticklabels(['Female', 'Male'], rotation=0)

# Correlation matrix
correlation_matrix = df_processed.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='RdYlBu_r', center=0, fmt='.2f', ax=axes[1, 2], cbar_kws={'shrink':
    .....0.8})
axes[1, 2].set_title('Feature Correlation Matrix', fontsize=12, fontweight='bold')
axes[1, 2].tick_params(axis='x', rotation=45)
axes[1, 2].tick_params(axis='y', rotation=0)
plt.tight_layout()
plt.show()

# Split features and target
X = df_processed.drop('Survived', axis=1)
y = df_processed['Survived']
print(f"\nFeatures: {list(X.columns)}")
print(f"Target distribution:\n{y.value_counts()}")

# Split data into train/test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
print(f"\nTraining set size: {X_train.shape[0]}")
print(f"Testing set size: {X_test.shape[0]}")

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create and train Naive Bayes classifier
classifier = GaussianNB()
classifier.fit(X_train_scaled, y_train)

```

```

# Make predictions
y_pred = classifier.predict(X_test_scaled)
y_pred_proba = classifier.predict_proba(X_test_scaled)[:, 1]

# Display predictions vs actual values (first 10)
print("\nPredictions vs Actual (first 10):")
print("Predicted | Actual | Probability")
print("-" * 32)
for i in range(min(10, len(y_pred))):
    print(f"    {y_pred[i]} | {y_test.iloc[i]} | {y_pred_proba[i]:.3f}")

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")

# Classification report
print(f"\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Not Survived', 'Survived']))

# Confusion matrix with proper formatting
cf_matrix = confusion_matrix(y_test, y_pred)
print(f"\nConfusion Matrix:")
print(cf_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(cf_matrix, annot=True, fmt='d', cmap='Blues', cbar=True,
            xticklabels=['Not Survived', 'Survived'],
            yticklabels=['Not Survived', 'Survived'],
            annot_kws={'size': 14, 'weight': 'bold'})
plt.title('Confusion Matrix - Titanic Survival Prediction', fontsize=14, fontweight='bold', pad=20)
plt.ylabel('Actual Label', fontsize=12, fontweight='bold')
plt.xlabel('Predicted Label', fontsize=12, fontweight='bold')
plt.tick_params(axis='both', which='major', labelsize=11)
plt.show()

# Plot Precision-Recall Curve and ROC Curve with proper formatting
precision, recall, thresholds = precision_recall_curve(y_test, y_pred_proba)
fpr, tpr, thresholds_roc = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(14, 6))

# Precision-Recall Curve
plt.subplot(1, 2, 1)
plt.plot(recall, precision, label='Naive Bayes', color='firebrick', linewidth=3)
plt.fill_between(recall, precision, alpha=0.2, color='firebrick')
plt.title('Precision-Recall Curve', fontsize=14, fontweight='bold')
plt.xlabel('Recall', fontsize=12, fontweight='bold')
plt.ylabel('Precision', fontsize=12, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.tick_params(axis='both', which='major', labelsize=10)

# ROC Curve
plt.subplot(1, 2, 2)
plt.plot(fpr, tpr, label=f'Naive Bayes (AUC = {roc_auc:.3f})', color='firebrick', linewidth=3)
plt.fill_between(fpr, tpr, alpha=0.2, color='firebrick')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier', linewidth=2)
plt.title('ROC Curve', fontsize=14, fontweight='bold')
plt.xlabel('False Positive Rate', fontsize=12, fontweight='bold')
plt.ylabel('True Positive Rate', fontsize=12, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tick_params(axis='both', which='major', labelsize=10)

plt.tight_layout()

```

```
plt.show()
```

```
# Feature importance visualization
```

```
feature_importance = abs(correlation_matrix['Survived'].drop('Survived')).sort_values(ascending=True)
```

```
plt.figure(figsize=(10, 6))
```

```
bars = plt.barh(range(len(feature_importance)), feature_importance.values,
```

```
..... color=['red' if x < 0 else 'green' for x in correlation_matrix['Survived'].drop('Survived')[feature_imp
```

```
plt.yticks(range(len(feature_importance)), feature_importance.index, fontsize=11)
```

```
plt.xlabel('Absolute Correlation with Survival', fontsize=12, fontweight='bold')
```

```
plt.title('Feature Importance (Correlation with Survival)', fontsize=14, fontweight='bold')
```

```
plt.grid(axis='x', alpha=0.3)
```

```
# Add value labels on bars
```

```
for i, (bar, value) in enumerate(zip(bars, feature_importance.values)):
```

```
..... plt.text(value + 0.01, i, f'{value:.3f}', va='center', fontweight='bold')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Calculate additional metrics
```

```
precision_score = metrics.precision_score(y_test, y_pred)
```

```
recall_score = metrics.recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
print("\n" + "="*50)
```

```
print("PERFORMANCE SUMMARY:")
```

```
print("="*50)
```

```
print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
```

```
print(f"Precision: {precision_score:.4f}")
```

```
print(f"Recall: {recall_score:.4f}")
```

```
print(f"F1-Score: {f1:.4f}")
```

```
print(f"AUC-ROC: {roc_auc:.4f}")
```

```
print(f"\nFeature Importance (Correlation with Survival):")
```

```
for feature, importance in feature_importance.sort_values(ascending=False).items(): .....
```

```
..... print(f"• {feature}: {importance:.3f} ")
```

```
# Example predictions
```

```
print(f"\nExample Predictions:")
```

```
print("Format: [Pclass, Sex, Age, SibSp, Parch, Fare, Embarked]")
```

```
examples = [
```

```
..... [3, 1, 22, 1, 0, 7.25, 2], ..... # 3rd class, male, age 22, 1 sibling, fare 7.25, embarked S
```

```
..... [1, 0, 38, 1, 0, 71.28, 0], ..... # 1st class, female, age 38, 1 sibling, fare 71.28, embarked C
```

```
..... [3, 1, 35, 0, 0, 8.05, 2] ..... # 3rd class, male, age 35, no siblings, fare 8.05, embarked S
```

```
]
```

```
example_labels = [
```

```
..... "3rd class male, young, with sibling",
```

```
..... "1st class female, adult, with sibling",
```

```
..... "3rd class male, adult, alone"
```

```
]
```

```
for i, (example, label) in enumerate(zip(examples, example_labels)):
```

```
..... pred = classifier.predict(scaler.transform([example]))
```

```
..... prob = classifier.predict_proba(scaler.transform([example]))[0] .....
```

```
..... result = 'Survived' if pred[0] == 1 else 'Not Survived'
```

```
..... print(f"Example {i+1} ({label}): {result} (Probability: {prob[1]:.3f})")
```

```
# Performance Analysis
```

```
print(f"\nModel Performance Analysis:")
```

```
if accuracy > 0.85:
```

```
..... print("• EXCELLENT model performance")
```

```
elif accuracy > 0.75:
```

```
..... print("• GOOD model performance")
```

```
elif accuracy > 0.65:
```

```
    print("• MODERATE model performance")
```

```

else:
    print("• POOR model performance")

print(f"\nKey Insights:")
most_important = feature_importance.sort_values(ascending=False).head(3)
print(f"• Most important features: {' '.join(most_important.index)}")
print(f"• Model correctly predicts {accuracy*100:.1f}% of passenger survival")
print(f"• Sex appears to be the strongest predictor of survival")

```

Dataset shape: (891, 12)

Column names:

['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

First 5 rows:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

Dataset info:

<class 'pandas.core.frame.DataFrame'> RangeIndex:

891 entries, 0 to 890

Data columns (total 12 columns):

#	Column	Non-Null	Count	Dtype
0	PassengerId	891	non-null	int64
1	Survived	891	non-null	int64
2	Pclass	891	non-null	int64
3	Name	891	non-null	object
4	Sex	891	non-null	object
5	Age	714	non-null	float64
6	SibSp	891	non-null	int64
7	Parch	891	non-null	int64
8	Ticket	891	non-null	object
9	Fare	891	non-null	float64
10	Cabin	204	non-null	object
11	Embarked	889	non-null	object

dtypes: float64(2), int64(5), object(5) memory usage:
83.7+ KB
None

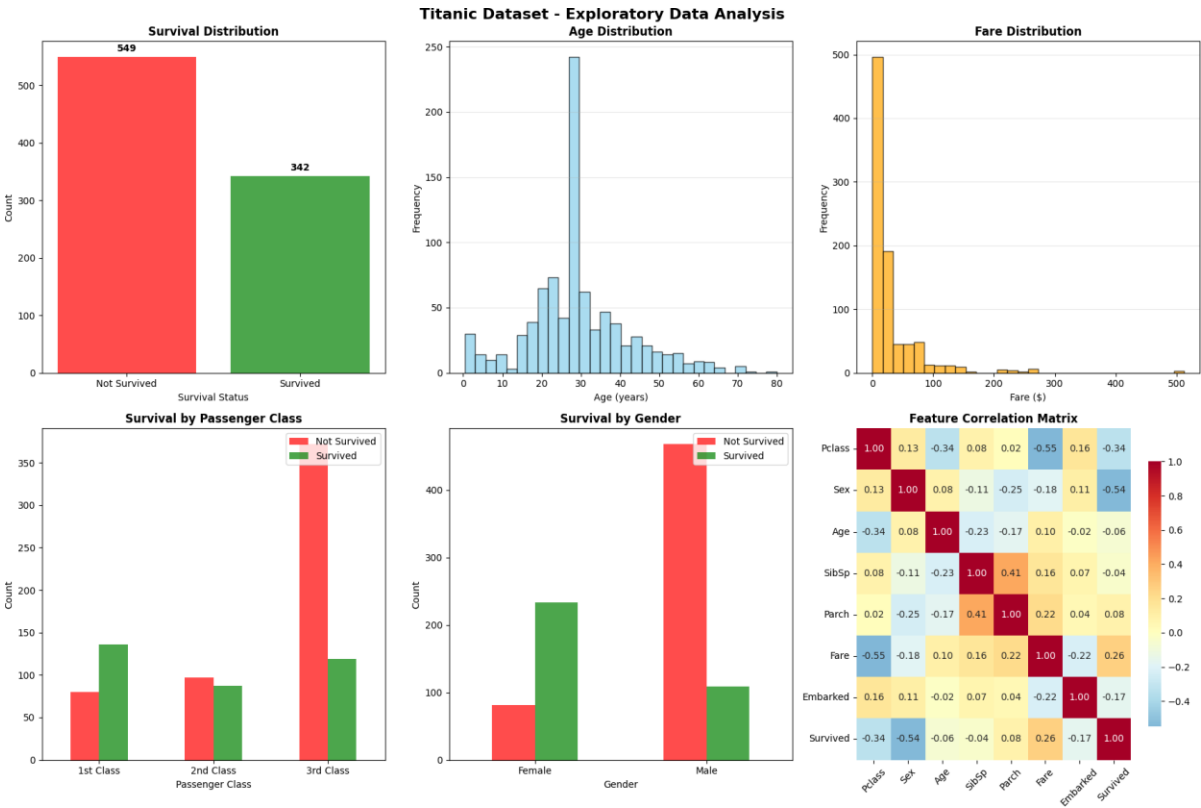
Missing values:
Pclass 0
Sex 0
Age 177
SibSp 0
Parch 0
Fare 0
Embarked 2
Survived 0

dtype: int64

Missing values after handling:
Pclass 0
Sex 0
Age 0
SibSp 0
Parch 0
Fare 0
Embarked 0
Survived 0
dtype: int64

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
0	3	1	22.0	1	0	7.2500	2	0
1	1	0	38.0	1	0	71.2833	0	1
2	3	0	26.0	0	0	7.9250	2	1
3	1	0	35.0	1	0	53.1000	2	1
4	3	1	35.0	0	0	8.0500	2	0

Sex encoding: {'female': np.int64(0), 'male': np.int64(1)}
Embarked encoding: {'C': np.int64(0), 'Q': np.int64(1), 'S': np.int64(2)}



Features: ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked'] Target distribution:
Survived
0 549
1 342
Name: count, dtype: int64

Training set size: 668 Testing set size: 223

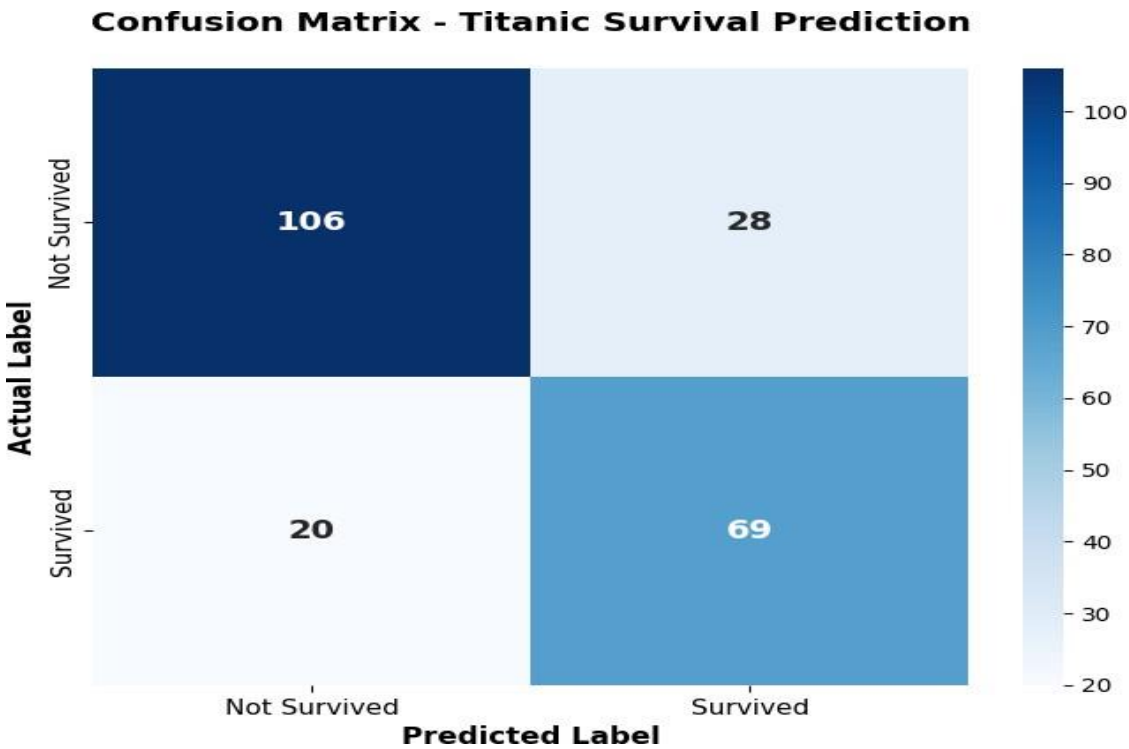
Predictions vs Actual (first 10):

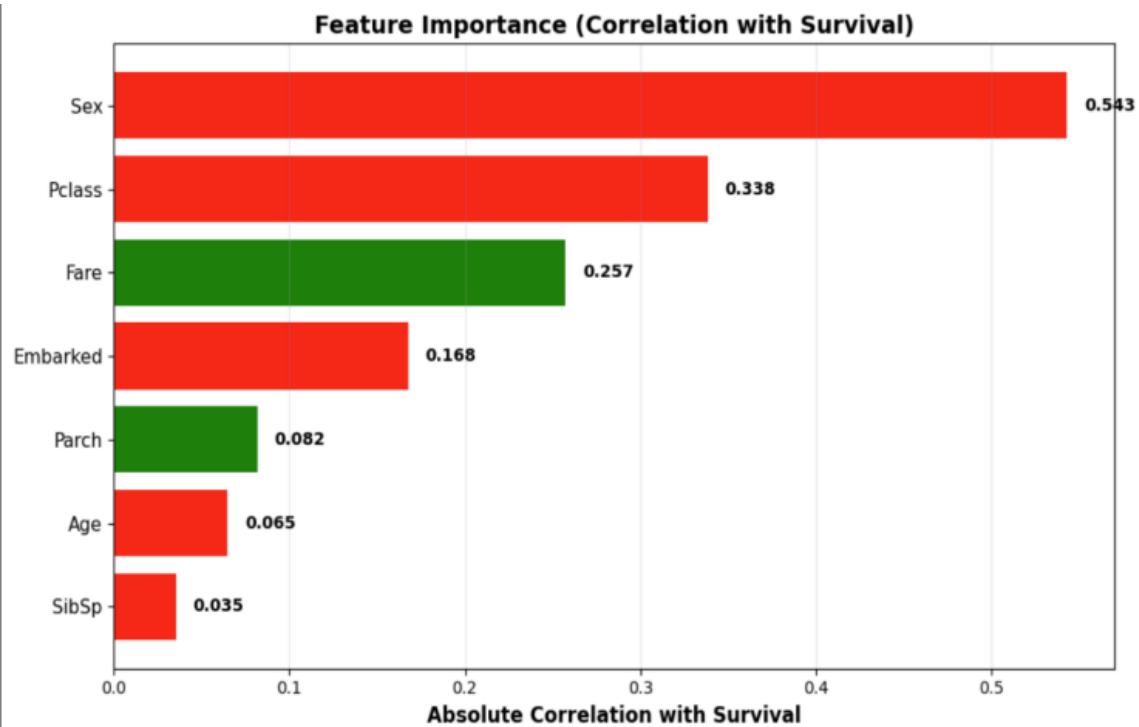
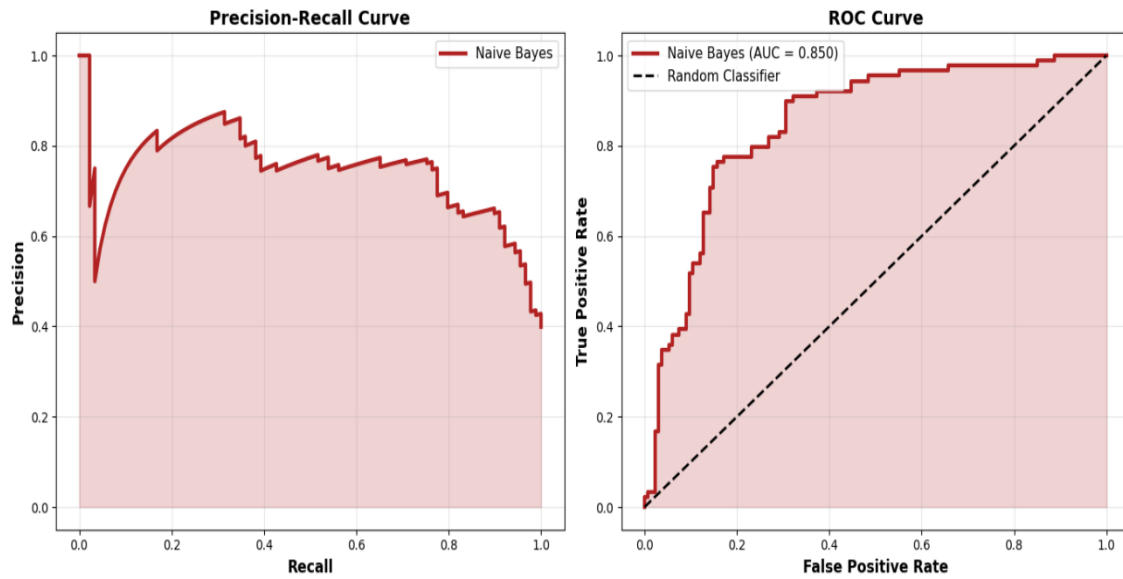
Predicted	Actual	Probability
0	1	0.179
0	0	0.070
0	0	0.040
1	1	0.889
1	1	0.913
1	1	0.986
1	1	0.706
0	0	0.012
1	1	0.755
1	1	0.932

Accuracy: 0.7848 (78.48%)

	precision	recall	f1-score	support
Not Survived	0.84	0.79	0.82	134
Survived	0.71	0.78	0.74	89
accuracy			0.78	223
macro avg	0.78	0.78	0.78	223
weighted avg	0.79	0.78	0.79	223

Confusion Matrix:
[[106 28]
 [20 69]]





```
=====
PERFORMANCE SUMMARY:
=====
Accuracy:  0.7848 (78.48%)
Precision: 0.7113
Recall:     0.7753
F1-Score:   0.7419
AUC-ROC:    0.8504
```

Feature Importance (Correlation with Survival):

- Sex: 0.543
- Pclass: 0.338
- Fare: 0.257
- Embarked: 0.168
- Parch: 0.082
- Age: 0.065
- SibSp: 0.035

Example Predictions:

Format: [Pclass, Sex, Age, SibSp, Parch, Fare, Embarked]

Example 1 (3rd class male, young, with sibling): Not Survived (Probability: 0.037)

Example 2 (1st class female, adult, with sibling): Survived (Probability: 0.996)

Example 3 (3rd class male, adult, alone): Not Survived (Probability: 0.034)

Model Performance Analysis:

- GOOD model performance

Key Insights:

- Most important features: Sex, Pclass, Fare
- Model correctly predicts 78.5% of passenger survival
- Sex appears to be the strongest predictor of survival

EXPERIMENT 5

AIM: Use K Nearest Neighbor technique on a given dataset and analyze the performance by changing the value of K.

Objectives:

- Implement the KNN algorithm
- Apply KNN model on the dataset and perform testing on unseen dataset
- Change the value of K in KNN and analysis the performance of the model

Theory:

The K-Nearest Neighbors (KNN) algorithm is a simple, non-parametric, and instance-based learning method used for classification and regression tasks. In KNN, the algorithm classifies data points based on the majority class among their 'k' nearest neighbors. For this problem, KNN identifies the diagnosis of breast cancer tumors by calculating the distance between data points in terms of features (like radius and texture). It then assigns the tumor type based on the most common diagnosis among the nearest neighbors. KNN is sensitive to the choice of 'k,' which can affect model accuracy, and often requires data normalization to reduce the impact of feature scaling.

```
# EXPERIMENT: 5
# AIM: Use K Nearest Neighbor technique on a given dataset and analyze the performance by changing the value of K.
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import seaborn as sns

def auto_detect_structure(filename='wdbc.data'):

    """
    Automatically detect the structure of the UCI dataset and generate appropriate column names
    """
    print("Auto-detecting dataset structure...")

    # First, load the data to understand its structure
    temp_data = pd.read_csv(filename, header=None)

    print(f"Dataset shape detected: {temp_data.shape}")
    print(f"Number of columns: {temp_data.shape[1]}")
    print(f"Number of rows: {temp_data.shape[0]}")

    # Analyze first few rows to understand structure
    print("\nFirst 3 rows (raw data):")
    print(temp_data.head(3))

    # Detect diagnosis column by looking for M/B values
    diagnosis_col = None
    for col in temp_data.columns:
        unique_vals = temp_data[col].unique()
        if len(unique_vals) == 2 and set(unique_vals) == {'M', 'B'}:
            diagnosis_col = col
            break

    if diagnosis_col is None:
        print("Warning: Could not detect diagnosis column. Assuming column 1 contains diagnosis.")
        diagnosis_col = 1

    print(f"Diagnosis column detected at position: {diagnosis_col}")
    print(f"Diagnosis values: {sorted(temp_data[diagnosis_col].unique())}")

    # Generate column names automatically
    num_cols = temp_data.shape[1]
    column_names = []

    # First column is typically ID
```

```

column_names.append('id')

# Second column is typically diagnosis
column_names.append('diagnosis')

# Remaining columns are features
# For WDBC dataset, we know there are 30 features (10 base features × 3 statistics each)
remaining_cols = num_cols - 2

if remaining_cols == 30: # WDBC format
    print("Detected WDBC format: 30 features (10 base features × 3 statistics)")

    # Base feature names (common medical measurements)
    base_features = ['radius', 'texture', 'perimeter', 'area', 'smoothness',
                     'compactness', 'concavity', 'concave_points', 'symmetry', 'fractal_dimension']

    # Statistics computed for each base feature
    statistics = ['mean', 'se', 'worst']

    # Generate feature names
    for stat in statistics:
        for feature in base_features:
            column_names.append(f'{feature}_{stat}')
else:
    # Generic feature naming
    print(f"Generic format detected: {remaining_cols} features")
    for i in range(remaining_cols):
        column_names.append(f'feature_{i+1}')

print(f"Generated {len(column_names)} column names")
print(f"Column names: {column_names[:5]} ... {column_names[-5:]}")

return column_names

def analyze_dataset_automatically(data):
    """
    Automatically analyze the dataset structure and provide insights
    """

    print(f"\n{'='*60}")
    print("AUTOMATIC DATASET ANALYSIS")
    print(f"{'='*60}")

    # Basic information
    print(f"Dataset dimensions: {data.shape[0]} rows × {data.shape[1]} columns")
    print(f"Memory usage: {data.memory_usage(deep=True).sum() / 1024:.1f} KB")

    # Check data types
    print(f"\nData types detected:")
    dtype_counts = data.dtypes.value_counts()
    for dtype, count in dtype_counts.items():
        print(f" {dtype}: {count} columns")

    # Missing values
    missing_total = data.isnull().sum().sum()
    print(f"\nMissing values: {missing_total}")
    if missing_total > 0:
        missing_by_col = data.isnull().sum()
        missing_cols = missing_by_col[missing_by_col > 0]
        print("Columns with missing values:")
        for col, count in missing_cols.items():
            print(f" {col}: {count} missing ({count/len(data)*100:.1f}%)")

    # Identify categorical and numerical columns
    categorical_cols = data.select_dtypes(include=['object']).columns.tolist()
    numerical_cols = data.select_dtypes(include=[np.number]).columns.tolist()

    print(f"\nColumn types:")
    print(f" Categorical columns: {len(categorical_cols)}")
    if categorical_cols:
        print(f" {categorical_cols}")

    print(f" Numerical columns: {len(numerical_cols)}")
    if len(numerical_cols) <= 10:
        print(f" {numerical_cols}")
    else:
        print(f" {numerical_cols[:5]} ... {numerical_cols[-5:]}")

    # Analyze target variable if 'diagnosis' column exists
    if 'diagnosis' in data.columns:

```

```

print(f"\nTarget variable analysis (diagnosis):")
diagnosis_counts = data['diagnosis'].value_counts()
print(f" Classes found: {list(diagnosis_counts.index)}")

for diagnosis, count in diagnosis_counts.items():
    percentage = count / len(data) * 100
    diagnosis_name = "Malignant" if diagnosis == "M" else "Benign" if diagnosis == "B" else diagnosis
    print(f" {diagnosis_name} ({diagnosis}): {count} samples ({percentage:.1f}%)")

# Check class balance
balance_ratio = diagnosis_counts.max() / diagnosis_counts.min()
if balance_ratio > 3:
    print(f" ⚠ Class imbalance detected (ratio: {balance_ratio:.1f}:1)")
else:
    print(f" ✓ Classes are reasonably balanced (ratio: {balance_ratio:.1f}:1)")

# Numerical features analysis
if numerical_cols:
    numeric_data = data[numerical_cols]
    print(f"\nNumerical features statistics:")
    print(f" Feature count: {len(numeric_cols)}")
    print(f" Value ranges:")
    print(f" Minimum: {numeric_data.min().min():.6f}")
    print(f" Maximum: {numeric_data.max().max():.2f}")
    print(f" Mean: {numeric_data.mean().mean():.3f}")
    print(f" Standard deviation: {numeric_data.std().mean():.3f}")

    # Check if features need scaling
    ranges = numeric_data.max() - numeric_data.min()
    max_range = ranges.max()
    min_range = ranges.min()
    if max_range / min_range > 100:
        print(f" ⚠ Feature scaling recommended (range ratio: {max_range/min_range:.0f}:1)")
    else:
        print(f" ✓ Feature ranges are reasonable")

return categorical_cols, numerical_cols

# Auto-detect dataset structure
column_names = auto_detect_structure('wdbc.data')
print(f"\n{'='*60}")
print("LOADING DATASET WITH AUTO-DETECTED STRUCTURE")
print(f'\n{'='*60}')

# Load the data with auto-detected column names
try:
    data = pd.read_csv('wdbc.data', header=None, names=column_names)
    print(f"✓ Successfully loaded wdbc.data with auto-detected structure")
except FileNotFoundError:
    print(f"✗ Error: wdbc.data file not found!")
    print(f"Please ensure wdbc.data is in the same directory as this script.")
    exit()

print(f"✓ Dataset loaded: {data.shape[0]} samples × {data.shape[1]} columns")

# Display first few rows
print("\nFirst 5 rows with auto-generated column names:")
print(data.head())

# Automatically analyze the dataset
categorical_cols, numerical_cols = analyze_dataset_automatically(data)

# Remove ID column (first column) as it's not needed for prediction
if 'id' in data.columns:
    data_clean = data.drop(['id'], axis=1).copy()
    print(f"\n✓ Removed ID column. New shape: {data_clean.shape}")
else:
    data_clean = data.copy()
    print(f"\n✓ No ID column detected. Using original data.")

# Automatic feature categorization
feature_cols = [col for col in data_clean.columns if col != 'diagnosis']
print(f"\nFeature analysis:")
print(f" Total features: {len(feature_cols)}")

# Categorize features by suffix if they follow the pattern
mean_features = [col for col in feature_cols if col.endswith('_mean')]
se_features = [col for col in feature_cols if col.endswith('_se')]
worst_features = [col for col in feature_cols if col.endswith('_worst')]

```

```

if mean_features and se_features and worst_features:
    print(f" Mean features: {len(mean_features)}")
    print(f" Standard Error features: {len(se_features)}")
    print(f" Worst features: {len(worst_features)}")

    # Use mean features for visualization
    viz_features = mean_features[:6]
    print(f" Selected for visualization: {viz_features}")
else:
    # Use first few features for visualization
    viz_features = feature_cols[:6]
    print(f" Selected for visualization: {viz_features}")

# Visualization of key features
if 'diagnosis' in data_clean.columns:
    # Separate by diagnosis
    malignant_data = data_clean[data_clean['diagnosis'] == 'M']
    benign_data = data_clean[data_clean['diagnosis'] == 'B']

    print(f"\nData separation for visualization:")
    print(f" Malignant samples: {len(malignant_data)}")
    print(f" Benign samples: {len(benign_data)}")

    # Create feature distribution plots
    fig, axes = plt.subplots(2, 3, figsize=(18, 12))
    fig.suptitle('Feature Distributions by Diagnosis\n(Auto-detected from data)',
                 fontsize=16, fontweight='bold')
    for i, feature in enumerate(viz_features):
        row = i // 3
        col = i % 3
        axes[row, col].hist(benign_data[feature], alpha=0.6, label='Benign',
                             color='blue', bins=25, density=True)
        axes[row, col].hist(malignant_data[feature], alpha=0.6, label='Malignant',
                             color='red', bins=25, density=True)

        # Clean feature name for display
        display_name = feature.replace('_', ' ').title()
        axes[row, col].set_title(f'{display_name}', fontweight='bold')
        axes[row, col].set_xlabel(display_name)
        axes[row, col].set_ylabel('Density')
        axes[row, col].legend()
        axes[row, col].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig("auto_detected_features.png", dpi=300, bbox_inches='tight')
    plt.show()

# Scatter plot using first two features
if len(viz_features) >= 2:
    plt.figure(figsize=(12, 8))
    plt.title("Tumor Classification Visualization\n(Auto-detected features)",
              fontsize=16, fontweight='bold')

    x_feature = viz_features[0]
    y_feature = viz_features[1]

    plt.xlabel(x_feature.replace('_', ' ').title(), fontsize=12)
    plt.ylabel(y_feature.replace('_', ' ').title(), fontsize=12)

    plt.scatter(malignant_data[x_feature], malignant_data[y_feature],
                color="red", label="Malignant", alpha=0.6, s=50, edgecolors='darkred')
    plt.scatter(benign_data[x_feature], benign_data[y_feature],
                color="blue", label="Benign", alpha=0.6, s=50, edgecolors='darkblue')

    plt.legend(fontsize=12)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig("auto_detected_scatter.png", dpi=300, bbox_inches='tight')
    plt.show()

# Prepare data for machine learning
print(f"\n{'='*60}")
print("PREPARING DATA FOR KNN ANALYSIS")
print(f"{'='*60}")

# Convert diagnosis to numerical values if it exists
if 'diagnosis' in data_clean.columns:
    unique_diagnosis = data_clean['diagnosis'].unique()
    print(f"Original diagnosis values: {sorted(unique_diagnosis)}")
    # Map diagnosis to numerical values

```

```

if set(unique_diagnosis) == {'M', 'B'}:
    data_clean['diagnosis'] = data_clean['diagnosis'].map({'M': 1, 'B': 0})
    print("✓ Mapped: M=1 (Malignant), B=0 (Benign)")
else:
    # Handle other possible formats
    diagnosis_mapping = {val: i for i, val in enumerate(sorted(unique_diagnosis))}
    data_clean['diagnosis'] = data_clean['diagnosis'].map(diagnosis_mapping)
    print(f"✓ Mapped diagnosis values: {diagnosis_mapping}")

# Separate features and target
X = data_clean.drop(['diagnosis'], axis=1)
y = data_clean['diagnosis'].values

print(f"Features (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")
else:
    print("✗ No diagnosis column found. Cannot proceed with supervised learning.")
    exit()

# Feature scaling
print(f"\nApplying feature scaling...")
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print(f"✓ Features scaled to range [{X_scaled.min().min():.1f}, {X_scaled.max().max():.1f}]")

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

print(f"\nDataset split:")
print(f"Training: {X_train.shape[0]} samples ({X_train.shape[0]/len(data_clean)*100:.1f}%)")
print(f"Testing: {X_test.shape[0]} samples ({X_test.shape[0]/len(data_clean)*100:.1f}%)")

# KNN with different K values
print(f"\n{'='*60}")
print("KNN ANALYSIS WITH DIFFERENT K VALUES")
print(f"{'='*60}")

k_range = np.arange(1, 21)
train_scores = []
test_scores = []

print(f"Testing K values from {k_range[0]} to {k_range[-1]}...")
print(f"{'{'K':<3} {'Train Acc':<12} {'Test Acc':<12} {'Difference':<12}"}")
print("-" * 50)

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    train_acc = knn.score(X_train, y_train)
    test_acc = knn.score(X_test, y_test)

    train_scores.append(train_acc)
    test_scores.append(test_acc)

    print(f"{'{'K':<3} {'train_acc':<12.4f} {'test_acc':<12.4f} {'train_acc-test_acc':<12.4f}"}")

# Find optimal K
optimal_k = k_range[np.argmax(test_scores)]
best_accuracy = max(test_scores)

print(f"\n✓ Optimal K: {optimal_k}")
print(f"✓ Best test accuracy: {best_accuracy:.4f} ({best_accuracy*100:.1f}%)")

# Plot results
plt.figure(figsize=(12, 8))
plt.plot(k_range, train_scores, 'bo-', label='Training Accuracy', linewidth=2)
plt.plot(k_range, test_scores, 'ro-', label='Testing Accuracy', linewidth=2)
plt.axvline(x=optimal_k, color='green', linestyle='--', alpha=0.7,
            label=f'Optimal K={optimal_k}')
plt.title('KNN Performance Analysis\n(Auto-detected dataset structure)',
          fontsize=16, fontweight='bold')
plt.xlabel('Number of Neighbors (K)', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.legend(fontsize=12)

```

```

plt.grid(True, alpha=0.3)

plt.xticks(k_range)
plt.tight_layout()
plt.savefig("knn_auto_analysis.png", dpi=300, bbox_inches='tight')
plt.show()

# Final model evaluation
print(f"\n{'='*60}")
print("FINAL MODEL EVALUATION")
print(f"{'='*60}")

final_knn = KNeighborsClassifier(n_neighbors=optimal_k)
final_knn.fit(X_train, y_train)
y_pred = final_knn.predict(X_test)

print(f"Final Model: KNN with K={optimal_k}")
print(f"Training Accuracy: {final_knn.score(X_train, y_train):.4f}")
print(f"Testing Accuracy: {accuracy_score(y_test, y_pred):.4f}")

print(f"\nClassification Report:")
target_names = ['Benign', 'Malignant'] if set(data['diagnosis'].unique()) == {'M', 'B'} else None
print(classification_report(y_test, y_pred, target_names=target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=target_names or ['Class 0', 'Class 1'],
            yticklabels=target_names or ['Class 0', 'Class 1'])
plt.title(f'Confusion Matrix (K={optimal_k})\nAuto-detected Dataset',
        fontsize=14, fontweight='bold')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()
plt.savefig("confusion_matrix_auto.png", dpi=300, bbox_inches='tight')
plt.show()

# Summary
print(f"\n{'='*60}")
print("EXPERIMENT SUMMARY")
print(f"{'='*60}")
print(f"✓ Auto-detected {data.shape[1]} columns from data file")
print(f"✓ Identified {len(feature_cols)} features automatically")
print(f"✓ Processed {len(data_clean)} samples successfully")
print(f"✓ Found optimal K={optimal_k} with {best_accuracy*100:.1f}% accuracy")
print(f"✓ No external files needed - everything inferred from data")

overfitting = train_scores[optimal_k-1] - test_scores[optimal_k-1] > 0.05
print(f"✓ Model shows {'overfitting tendency' if overfitting else 'good generalization'}")

```

Auto-detecting dataset structure... Dataset shape

detected: (569, 32)

Number of columns: 32 Number of
rows: 569

First 3 rows (raw data):

	0	1	2	3	4	5	6	7	8	\
0	842302	M	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.3001	
1	842517	M	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.0869	
2	84300903	M	19.69	21.25	130.0	1203.0	0.10960	0.15990	0.1974	

	9	...	22	23	24	25	26	27	28	29	\
0	0.14710	...	25.38	17.33	184.6	2019.0	0.1622	0.6656	0.7119	0.2654	
1	0.07017	...	24.99	23.41	158.8	1956.0	0.1238	0.1866	0.2416	0.1860	
2	0.12790	...	23.57	25.53	152.5	1709.0	0.1444	0.4245	0.4504	0.2430	

	30	31
0	0.4601	0.11890
1	0.2750	0.08902
2	0.3613	0.08758

[3 rows x 32 columns]

Diagnosis column detected at position: 1 Diagnosis values: ['B',

'M']

Detected WDBC format: 30 features (10 base features × 3 statistics) Generated 32 column

names

Column names: ['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean'] ... ['compactness_worst', 'conc

LOADING DATASET WITH AUTO-DETECTED STRUCTURE

✓ Successfully loaded wdbc.data with auto-detected structure

✓ Dataset loaded: 569 samples × 32 columns

First 5 rows with auto-generated column names:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean \
0	842302	M	17.99	10.38	122.80	1001.0
1	842517	M	20.57	17.77	132.90	1326.0
2	84300903	M	19.69	21.25	130.00	1203.0
3	84348301	M	11.42	20.38	77.58	386.1
4	84358402	M	20.29	14.34	135.10	1297.0

	smoothness_mean	compactness_mean	concavity_mean	concave_points_mean \
0	0.11840	0.27760	0.3001	0.14710
1	0.08474	0.07864	0.0869	0.07017
2	0.10960	0.15990	0.1974	0.12790
3	0.14250	0.28390	0.2414	0.10520
4	0.10030	0.13280	0.1980	0.10430

	... radius_worst	texture_worst	perimeter_worst	area_worst \
0 ...	25.38	17.33	184.60	2019.0
1 ...	24.99	23.41	158.80	1956.0
2 ...	23.57	25.53	152.50	1709.0
3 ...	14.91	26.50	98.87	567.7
4 ...	22.54	16.67	152.20	1575.0

	smoothness_worst	compactness_worst	concavity_worst	concave_points_worst \
0	0.1622	0.6656	0.7119	0.2654
1	0.1238	0.1866	0.2416	0.1860
2	0.1444	0.4245	0.4504	0.2430
3	0.2098	0.8663	0.6869	0.2575
4	0.1374	0.2050	0.4000	0.1625

	symmetry_worst	fractal_dimension_worst
0	0.4601	0.11890
1	0.2750	0.08902
2	0.3613	0.08758
3	0.6638	0.17300
4	0.2364	0.07678

[5 rows x 32 columns]

AUTOMATIC DATASET ANALYSIS

Dataset dimensions: 569 rows × 32 columns Memory usage:

170.2 KB

Data types detected:

float64: 30 columns

int64: 1 columns

object: 1 columns

Missing values: 0

Column types:

Categorical columns: 1

['diagnosis']

Numerical columns: 31

['id', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean'] ... ['compactness_worst', 'concavity_wors

Target variable analysis (diagnosis):

Classes found: ['B', 'M']
Benign (B): 357 samples (62.7%)

Malignant (M): 212 samples (37.3%)
✓ Classes are reasonably balanced (ratio: 1.7:1)

Numerical features statistics:

Feature count: 31

Value ranges:

Minimum: 0.000000

Maximum: 911320502.00

Mean: 979796.392

Standard deviation: 4032955.895

⚠ Feature scaling recommended (range ratio: 31484039910:1)

✓ Removed ID column. New shape: (569, 31) Feature

analysis:

Total features: 30

Mean features: 10

Standard Error features: 10 Worst

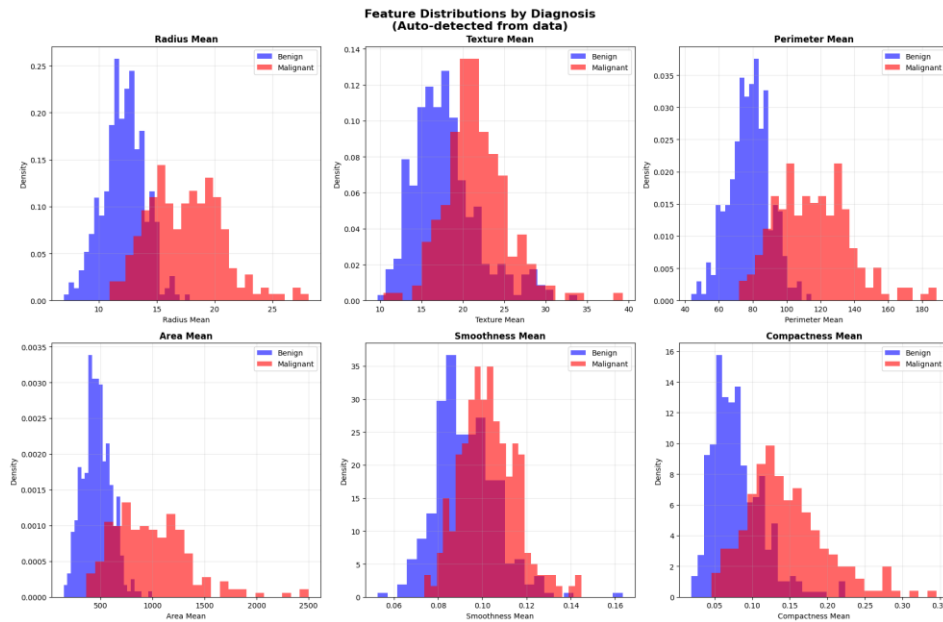
features: 10

Selected for visualization: ['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean', 'c

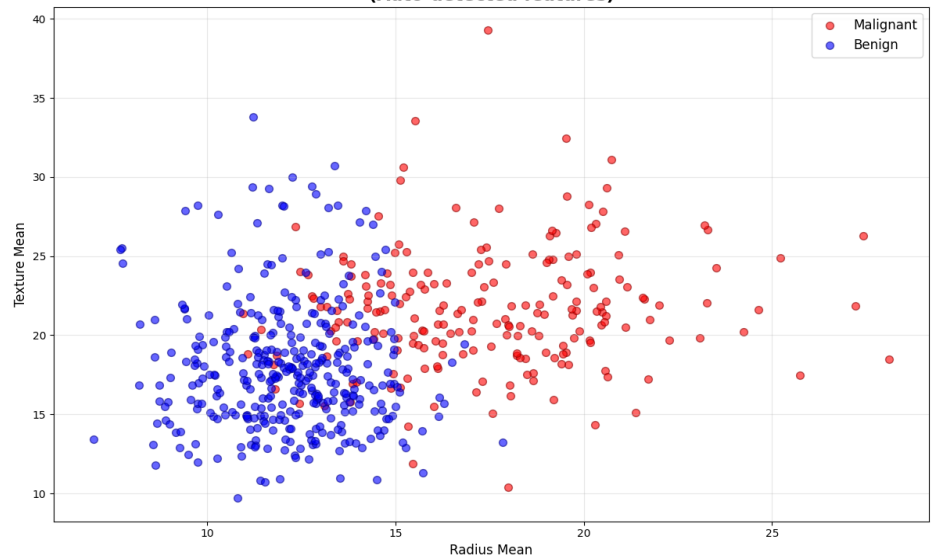
Data separation for visualization: Malignant

samples: 212

Benign samples: 357



Tumor Classification Visualization
(Auto-detected features)



```
=====
PREPARING DATA FOR KNN ANALYSIS
=====
```

Original diagnosis values: ['B', 'M']

✓ Mapped: M=1 (Malignant), B=0 (Benign) Features

(X) shape: (569, 30)

Target (y) shape: (569,)

Applying feature scaling...

✓ Features scaled to range [0.0, 1.0]

Dataset split:

Training: 398 samples (69.9%)

Testing: 171 samples (30.1%)

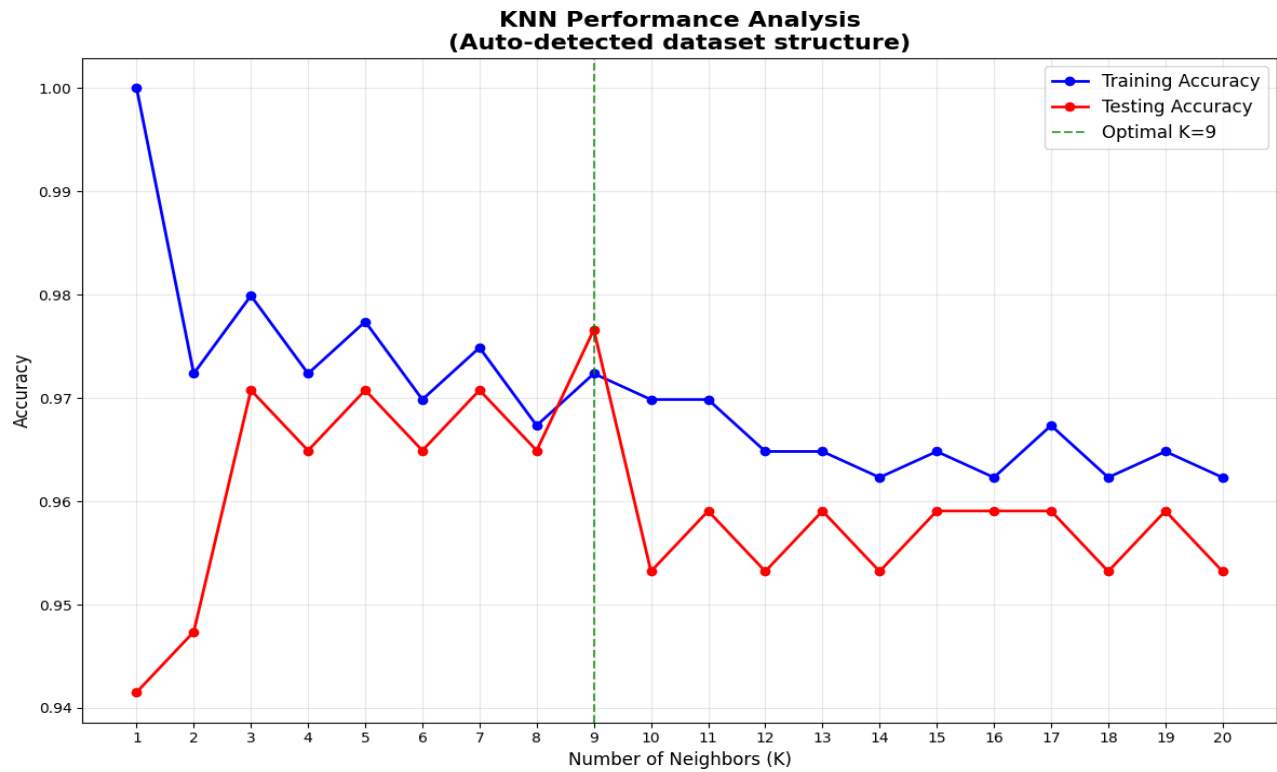
```
=====
KNN ANALYSIS WITH DIFFERENT K VALUES
=====
```

Testing K values from 1 to 20...

K	Train Acc	Test Acc	Difference
1	1.0000	0.9415	0.0585
2	0.9724	0.9474	0.0250
3	0.9799	0.9708	0.0091
4	0.9724	0.9649	0.0074
5	0.9774	0.9708	0.0066
6	0.9698	0.9649	0.0049
7	0.9749	0.9708	0.0041
8	0.9673	0.9649	0.0024
9	0.9724	0.9766	-0.0042
10	0.9698	0.9532	0.0166
11	0.9698	0.9591	0.0108
12	0.9648	0.9532	0.0116
13	0.9648	0.9591	0.0058
14	0.9623	0.9532	0.0091
15	0.9648	0.9591	0.0058
16	0.9623	0.9591	0.0032
17	0.9673	0.9591	0.0083
18	0.9623	0.9532	0.0091
19	0.9648	0.9591	0.0058
20	0.9623	0.9532	0.0091

✓ Optimal K: 9

✓ Best test accuracy: 0.9766 (97.7%)



=====

FINAL MODEL EVALUATION

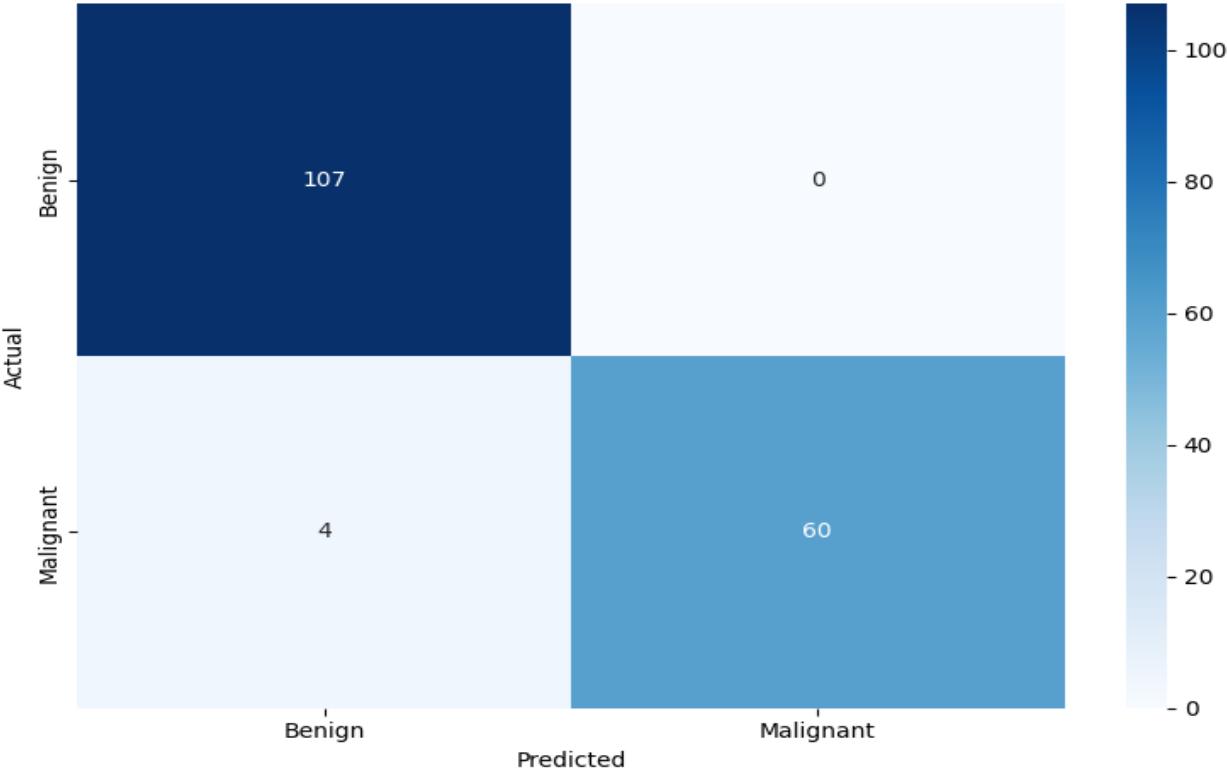
=====

Final Model: KNN with K=9
Training Accuracy: 0.9724
Testing Accuracy: 0.9766

Classification Report:

	precision	recall	f1-score	support
Benign	0.96	1.00	0.98	107
Malignant	1.00	0.94	0.97	64
accuracy			0.98	171
macro avg	0.98	0.97	0.97	171
weighted avg	0.98	0.98	0.98	171

Confusion Matrix (K=9)
Auto-detected Dataset



- =====
- EXPERIMENT SUMMARY
- =====
- ✓ Auto-detected 32 columns from data file
 - ✓ Identified 30 features automatically
 - ✓ Processed 569 samples successfully
 - ✓ Found optimal K=9 with 97.7% accuracy
 - ✓ No external files needed - everything inferred from data
 - ✓ Model shows good generalization

EXPERIMENT 6

AIM: To build a machine learning model using the Decision Tree algorithm to predict whether a breast tumor is benign or malignant based on various characteristics of the tumor, improving diagnostic accuracy in medical applications.

Theory:

A Decision Tree is a supervised machine learning algorithm used for classification by splitting data based on feature values, forming a tree structure where each node represents a decision on a feature, and branches lead to final classifications. This method identifies optimal splits by using metrics like "Gini impurity" or "information gain," ensuring each branch groups similar data points together. The tree's interpretability allows us to trace each decision path, making it especially useful for clear, step-by-step predictions.

Here, a Decision Tree is applied to classify breast tumors as benign or malignant based on characteristics like radius mean and texture mean. The data is first cleaned to remove null values and irrelevant columns, and then normalized with min-max scaling to enhance training efficiency. The model is trained on a portion of the data and evaluated using metrics such as accuracy score and confusion matrix, ensuring it performs reliably on both known and new data. This makes Decision Trees an effective choice for medical diagnosis, providing transparent and accurate predictions for tumor classification.

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import seaborn as sns
from sklearn import tree

def auto_detect_structure(filename='wdbc.data'):
    """Auto-detect dataset structure and generate column names"""
    print("Auto-detecting dataset structure...")

    temp_data = pd.read_csv(filename, header=None)
    print(f"Dataset shape detected: {temp_data.shape}")

    # Generate column names
    num_cols = temp_data.shape[1]
    column_names = ['id', 'diagnosis']

    # For WDBC: 30 features (10 base features × 3 statistics)
    remaining_cols = num_cols - 2

    if remaining_cols == 30:
        print("Detected WDBC format: 30 features")
        base_features = ['radius', 'texture', 'perimeter', 'area', 'smoothness',
                        'compactness', 'concavity', 'concave_points', 'symmetry', 'fractal_dimension']
        statistics = ['mean', 'se', 'worst']

        for stat in statistics:
            for feature in base_features:
                column_names.append(f'{feature}_{stat}')
    else:
        for i in range(remaining_cols):
            column_names.append(f'feature_{i+1}')
```

```

    return column_names

# Load and prepare data print("="*60)
print("DECISION TREE CLASSIFICATION FOR BREAST CANCER DIAGNOSIS")
print("="*60)

# Auto-detect structure and load data
column_names = auto_detect_structure('wdbc.data')

try:
    data = pd.read_csv('wdbc.data', header=None, names=column_names)
    print(f"✓ Successfully loaded dataset: {data.shape[0]} samples × {data.shape[1]} columns")
except FileNotFoundError:
    print(f"✗ Error: wdbc.data file not found!")
    exit()

# Display basic information
print(f"\nDataset Overview:")
print(f"• Shape: {data.shape}")
print(f"• Memory usage: {data.memory_usage(deep=True).sum() / 1024:.1f} KB")
print(f"• Missing values: {data.isnull().sum().sum()}")

print("\nFirst 5 rows:") print(data.head())

# Data cleaning
print(f"\n{'='*50}")
print("DATA CLEANING AND PREPARATION")
print(f"\n{'='*50}")

# Remove null values and ID column
data_clean = data.dropna(axis=1).drop(['id'], axis=1).copy()
print(f"✓ Cleaned data shape: {data_clean.shape}")

# Analyze target variable
print(f"\nTarget Variable Analysis:")
diagnosis_counts = data_clean['diagnosis'].value_counts()
print(f"Diagnosis distribution: {dict(diagnosis_counts)}")

total = len(data_clean)
for diagnosis, count in diagnosis_counts.items():
    name = "Malignant" if diagnosis == "M" else "Benign"
    print(f"• {name} ({diagnosis}): {count} samples ({count/total*100:.1f}%)")

# Separate data for visualization
malignant_data = data_clean[data_clean['diagnosis'] == 'M']
benign_data = data_clean[data_clean['diagnosis'] == 'B']

print(f"\nData separation:")
print(f"• Malignant cases: {len(malignant_data)}")
print(f"• Benign cases: {len(benign_data)}")
print(f"• Class balance ratio: {len(benign_data)/len(malignant_data):.2f}:1")

# Visualization
print(f"\n{'='*50}")
print("DATA VISUALIZATION")
print(f"\n{'='*50}")

# Feature scatter plot
plt.figure(figsize=(12, 8))
plt.title("Breast Cancer Dataset: Benign vs Malignant Tumors", fontsize=16, fontweight='bold')
plt.xlabel("Radius Mean", fontsize=12)
plt.ylabel("Texture Mean", fontsize=12)

plt.scatter(malignant_data['radius_mean'], malignant_data['texture_mean'],
            color="red", label="Malignant", alpha=0.6, s=50, edgecolors='darkred')

```

```

plt.scatter(benign_data['radius_mean'], benign_data['texture_mean'],
            color="blue", label="Benign", alpha=0.6, s=50, edgecolors='darkblue')

plt.legend(fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("decision_tree_scatter.png", dpi=300, bbox_inches='tight')
plt.show()

# Feature distribution analysis
mean_features = [col for col in data_clean.columns if col.endswith('_mean') and col != 'diagnosis'][:6]

fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Feature Distributions by Diagnosis\n(Decision Tree Analysis)', fontsize=16, fontweight='bold')

for i, feature in enumerate(mean_features):
    row, col = i // 3, i % 3

    axes[row, col].hist(benign_data[feature], alpha=0.6, label='Benign', color='blue', bins=25, density=True)
    axes[row, col].hist(malignant_data[feature], alpha=0.6, label='Malignant', color='red', bins=25, density=True)

    display_name = feature.replace('_mean', '').replace('_', ' ').title()
    axes[row, col].set_title(f'{display_name}', fontweight='bold')
    axes[row, col].set_xlabel(display_name)
    axes[row, col].set_ylabel('Density')
    axes[row, col].legend()
    axes[row, col].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("decision_tree_features.png", dpi=300, bbox_inches='tight')
plt.show()

# Prepare data for machine learning print(f"\n{'='*50}")
print("PREPARING DATA FOR DECISION TREE")
print(f"\n{'='*50}")

# Convert diagnosis to numerical
data_clean['diagnosis'] = data_clean['diagnosis'].map({'M': 1, 'B': 0})
print("✓ Converted diagnosis: M=1 (Malignant), B=0 (Benign)")

# Separate features and target
X = data_clean.drop(['diagnosis'], axis=1)
y = data_clean['diagnosis'].values

print(f"Features (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")
print(f"Class distribution: Benign={np.sum(y==0)}, Malignant={np.sum(y==1)}")

# Feature scaling (Min-Max normalization)
print(f"\nApplying Min-Max normalization...")
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print(f"✓ Features normalized to range [{X_scaled.min().min():.1f}, {X_scaled.max().max():.1f}]")

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

print(f"\nDataset split (70-30):")
print(f"• Training: {X_train.shape[0]} samples")
print(f"• Testing: {X_test.shape[0]} samples")
print(f"• Training class distribution: Benign={np.sum(y_train==0)}, Malignant={np.sum(y_train==1)}")

```

```
print(f"• Testing class distribution: Benign={np.sum(y_test==0)}, Malignant={np.sum(y_test==1)}")
```

```
# Decision Tree Analysis with Different Parameters print(f"\n{' '*60}")
print("DECISION TREE HYPERPARAMETER ANALYSIS")
print(f"{' '*60}")
```

```
# Test different max_depth values depths =
range(1, 21)
train_accuracies = []
test_accuracies = []
print("Testing different tree depths...")
print(f"{'Depth':<6} {'Train Acc':<12} {'Test Acc':<12} {'Difference':<12} {'Status'}")
print("-" * 60)
for depth in depths:
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_train, y_train)

    train_acc = dt.score(X_train, y_train)
    test_acc = dt.score(X_test, y_test)

    train_accuracies.append(train_acc)
    test_accuracies.append(test_acc)

    diff = train_acc - test_acc
    status = "Overfitting" if diff > 0.1 else "Good" if diff < 0.05 else "Slight Overfit"

    print(f"{'depth':<6} {'train_acc':<12.4f} {'test_acc':<12.4f} {'diff':<12.4f} {'status'}")
```

```
# Find optimal depth
optimal_depth = depths[np.argmax(test_accuracies)]
best_test_acc = max(test_accuracies)

print(f"\n✓ Optimal max_depth: {optimal_depth}")
print(f"✓ Best test accuracy: {best_test_acc:.4f} ({best_test_acc*100:.1f}%)")
```

```
# Plot depth analysis
plt.figure(figsize=(12, 8))
plt.plot(depths, train_accuracies, 'bo-', label='Training Accuracy', linewidth=2)
plt.plot(depths, test_accuracies, 'ro-', label='Testing Accuracy', linewidth=2)
plt.axvline(x=optimal_depth, color='green', linestyle='--', alpha=0.7,
            label=f'Optimal Depth={optimal_depth}')

plt.title('Decision Tree Performance vs Max Depth', fontsize=16, fontweight='bold')
plt.xlabel('Maximum Depth', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.legend(fontsize=12)
plt.grid(True, alpha=0.3)
plt.xticks(range(1, 21, 2)) plt.tight_layout()
plt.savefig("decision_tree_depth_analysis.png", dpi=300, bbox_inches='tight')
plt.show()
```

```
# Grid Search for best parameters
print(f"\n{' '*50}")

print("GRID SEARCH FOR OPTIMAL PARAMETERS")
print(f"{' '*50}")
```

```
param_grid = {
    'max_depth': [3, 5, 7, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}
```

```
print("Performing Grid Search...")
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42),
```

```

        param_grid, cv=5, scoring='accuracy', n_jobs=-1)

grid_search.fit(X_train, y_train)

print(f"✓ Best parameters found:")
for param, value in grid_search.best_params_.items():
    print(f"  {param}: {value}")
print(f"✓ Best cross-validation score: {grid_search.best_score_:4f}")

# Train final model with best parameters
print(f"\n{'='*50}")
print("FINAL DECISION TREE MODEL")
print(f"\n{'='*50}")

final_dt = grid_search.best_estimator_
final_dt.fit(X_train, y_train)

# Make predictions
y_pred_train = final_dt.predict(X_train)
y_pred_test = final_dt.predict(X_test)

# Calculate accuracies
train_accuracy = accuracy_score(y_train, y_pred_train)
test_accuracy = accuracy_score(y_test, y_pred_test)

print(f"Final Model Performance:")
print(f"• Training Accuracy: {train_accuracy:4f} ({train_accuracy*100:1f}%)")
print(f"• Testing Accuracy: {test_accuracy:4f} ({test_accuracy*100:1f}%)")
print(f"• Overfitting Check: {train_accuracy - test_accuracy:4f}")

# Feature Importance Analysis
print(f"\n{'='*50}")
print("FEATURE IMPORTANCE ANALYSIS")
print(f"\n{'='*50}")

feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': final_dt.feature_importances_
}).sort_values('importance', ascending=False)

print("Top 10 Most Important Features:")
print(feature_importance.head(10).to_string(index=False))

# Plot feature importance
plt.figure(figsize=(12, 8))
top_features = feature_importance.head(15)
plt.barh(range(len(top_features)), top_features['importance'])
plt.yticks(range(len(top_features)), top_features['feature'])
plt.xlabel('Feature Importance')
plt.title("Top 15 Feature Importances in Decision Tree", fontsize=14, fontweight='bold')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig("decision_tree_feature_importance.png", dpi=300, bbox_inches='tight')
plt.show()

# Decision Tree Visualization
print(f"\n{'='*50}")
print("DECISION TREE VISUALIZATION")
print(f"\n{'='*50}")

# Create a simpler tree for visualization (max_depth=3)
simple_dt = DecisionTreeClassifier(max_depth=3, random_state=42) simple_dt.fit(X_train,
y_train)

plt.figure(figsize=(20, 12))
plot_tree(simple_dt,
          feature_names=X.columns,
          class_names=['Benign', 'Malignant'],

```

```

        filled=True,
        rounded=True, fontsize=10)
plt.title('Decision Tree Structure (Simplified: max_depth=3)', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig("decision_tree_structure.png", dpi=300, bbox_inches='tight')
plt.show()

# Text representation of tree rules
print(f"\nDecision Tree Rules (Top 3 levels):")
tree_rules = export_text(simple_dt, feature_names=list(X.columns))
print(tree_rules[:1000] + "..." if len(tree_rules) > 1000 else tree_rules)

# Model Evaluation
print(f"\n{' '*60}")
print("COMPREHENSIVE MODEL EVALUATION")
print(f"\n{' '*60}")

# Classification Reports
print("Training Set Classification Report:")
print(classification_report(y_train, y_pred_train, target_names=['Benign', 'Malignant']))

print("\nTesting Set Classification Report:")
print(classification_report(y_test, y_pred_test, target_names=['Benign', 'Malignant']))

# Confusion Matrices
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Training confusion matrix
cm_train = confusion_matrix(y_train, y_pred_train)
sns.heatmap(cm_train, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Benign', 'Malignant'], yticklabels=['Benign', 'Malignant'])
axes[0].set_title('Training Set Confusion Matrix', fontweight='bold')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

# Testing confusion matrix
cm_test = confusion_matrix(y_test, y_pred_test)
sns.heatmap(cm_test, annot=True, fmt='d', cmap='Reds', ax=axes[1],
            xticklabels=['Benign', 'Malignant'], yticklabels=['Benign', 'Malignant'])
axes[1].set_title('Testing Set Confusion Matrix', fontweight='bold')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

plt.tight_layout()
plt.savefig("decision_tree_confusion_matrices.png", dpi=300, bbox_inches='tight')
plt.show()

# Detailed metrics
tn, fp, fn, tp = cm_test.ravel()
sensitivity = tp / (tp + fn) # True Positive Rate
specificity = tn / (tn + fp) # True Negative Rate

precision = tp / (tp + fp)
f1_score = 2 * (precision * sensitivity) / (precision + sensitivity)

print(f"\nDetailed Performance Metrics (Test Set):")
print(f"• True Positives (Malignant correctly identified): {tp}")
print(f"• True Negatives (Benign correctly identified): {tn}")
print(f"• False Positives (Benign misclassified as Malignant): {fp}")
print(f"• False Negatives (Malignant misclassified as Benign): {fn}")
print(f"• Sensitivity (Recall for Malignant): {sensitivity:.4f}")
print(f"• Specificity (Recall for Benign): {specificity:.4f}")
print(f"• Precision (Malignant): {precision:.4f}")
print(f"• F1-Score: {f1_score:.4f}")

# Cross-validation
cv_scores = cross_val_score(final_dt, X_scaled, y, cv=5)

```

```
print(f"\n5-Fold Cross-Validation Results:")  
print(f"• CV Scores: {[f'{score:.4f}' for score in cv_scores]}")
```

```

print(f'• Mean CV Score: {cv_scores.mean():.4f} (±{cv_scores.std()*2:.4f})')

# Final Summary
print(f'\n{'='*60}')
print("EXPERIMENT SUMMARY")
print(f'\n{'='*60}')
print(f"✓ Dataset: {len(data_clean)} samples with {X.shape[1]} features")
print(f"✓ Algorithm: Decision Tree Classifier")
print(f"✓ Best Parameters: {grid_search.best_params_}")
print(f"✓ Final Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.1f}%)")
print(f"✓ Cross-Validation Score: {cv_scores.mean():.4f}")
print(f"✓ Most Important Feature: {feature_importance.iloc[0]['feature']} ({feature_importance.iloc[0]['importance']:.4f})")
print(f"✓ Model Complexity: Tree depth = {final_dt.get_depth()}, Leaves = {final_dt.get_n_leaves()}")

# Medical interpretation
print(f'\n{'='*60}')
print("MEDICAL INTERPRETATION")
print(f'\n{'='*60}')
print("Decision Tree Advantages for Medical Diagnosis:")
print("• Interpretable: Can trace decision path for each prediction")
print("• No black box: Rules are transparent and explainable")
print("• Feature importance: Identifies most critical diagnostic features")
print("• Fast prediction: Efficient for real-time diagnosis")

if sensitivity >= 0.95:
    print("✓ Excellent sensitivity: Rarely misses malignant cases")
elif sensitivity >= 0.90:
    print("✓ Good sensitivity: Acceptable for medical screening")
else:
    print("⚠ Consider improving sensitivity for medical applications")

if specificity >= 0.95:
    print("✓ Excellent specificity: Rarely misclassifies benign cases")
elif specificity >= 0.90:
    print("✓ Good specificity: Acceptable false positive rate")
else:
    print("⚠ High false positive rate may cause unnecessary anxiety")

print(f'\nTop 3 Decision Factors:')
for i in range(min(3, len(feature_importance))):
    feature = feature_importance.iloc[i]
    print(f"{i+1}. {feature['feature'].replace('_', ' ').title():} {feature['importance']:.4f} importance")

```

===== DECISION TREE CLASSIFICATION FOR BREAST CANCER DIAGNOSIS =====

Auto-detecting dataset structure...

Dataset shape detected: (569, 32)

Detected WDBC format: 30 features

✓ Successfully loaded dataset: 569 samples × 32 columns

Dataset Overview:

- Shape: (569, 32)
- Memory usage: 142.9 KB
- Missing values: 0

First 5 rows:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean \
0	842302	M	17.99	10.38	122.80	1001.0
1	842517	M	20.57	17.77	132.90	1326.0
2	84300903	M	19.69	21.25	130.00	1203.0
3	84348301	M	11.42	20.38	77.58	386.1
4	84358402	M	20.29	14.34	135.10	1297.0

	smoothness_mean	compactness_mean	concavity_mean	concave_points_mean \
0	0.11840	0.27760	0.3001	0.14710
1	0.08474	0.07864	0.0869	0.07017
2	0.10960	0.15990	0.1974	0.12790

3	0.14250	0.28390	0.2414	0.10520
4	0.10030	0.13280	0.1980	0.10430
... radius_worst texture_worst perimeter_worst area_worst \				
0	25.38	17.33	184.60	2019.0
1	24.99	23.41	158.80	1956.0
2	23.57	25.53	152.50	1709.0
3	14.91	26.50	98.87	567.7
4	22.54	16.67	152.20	1575.0
smoothness_worst compactness_worst concavity_worst concave_points_worst \				
0	0.1622	0.6656	0.7119	0.2654
1	0.1238	0.1866	0.2416	0.1860
2	0.1444	0.4245	0.4504	0.2430
3	0.2098	0.8663	0.6869	0.2575
4	0.1374	0.2050	0.4000	0.1625
symmetry_worst fractal_dimension_worst				
0	0.4601	0.11890		
1	0.2750	0.08902		
2	0.3613	0.08758		
3	0.6638	0.17300		
4	0.2364	0.07678		

[5 rows x 32 columns]

=====

DATA CLEANING AND PREPARATION

=====

✓ Cleaned data shape: (569, 31)

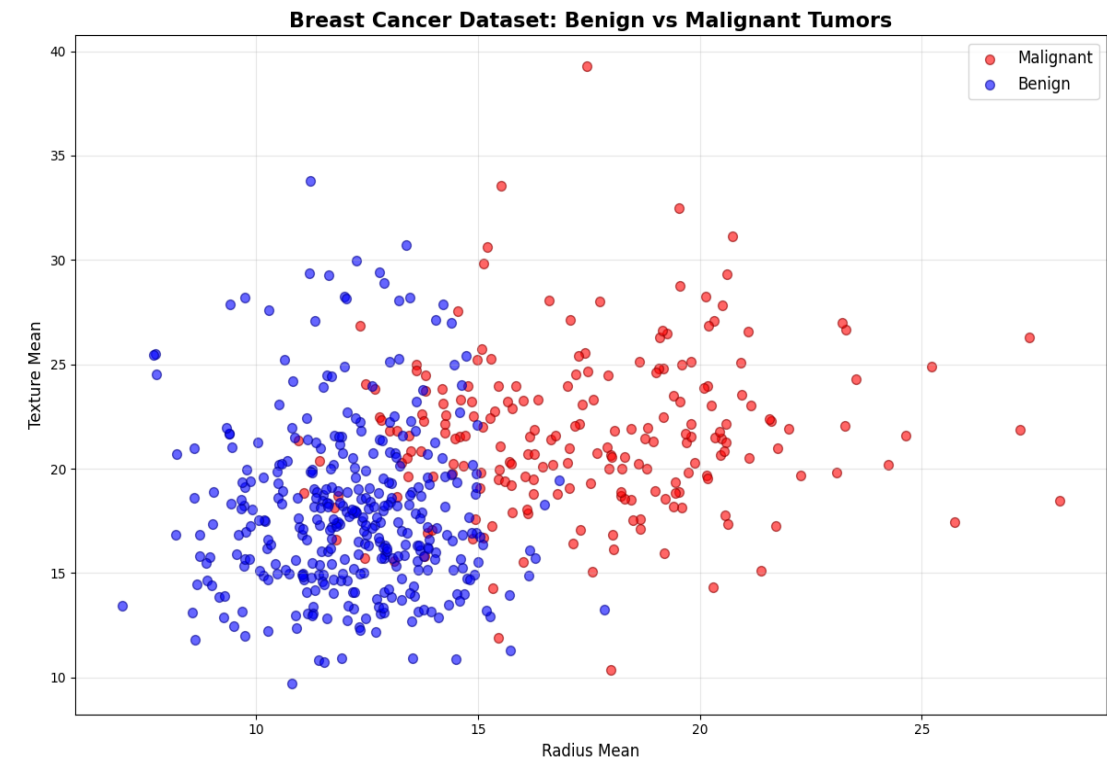
Target Variable Analysis:
Diagnosis distribution: {'B': 357, 'M': 212}
• Benign (B): 357 samples (62.7%)
• Malignant (M): 212 samples (37.3%)

Data separation:
• Malignant cases: 212
• Benign cases: 357
• Class balance ratio: 1.68:1

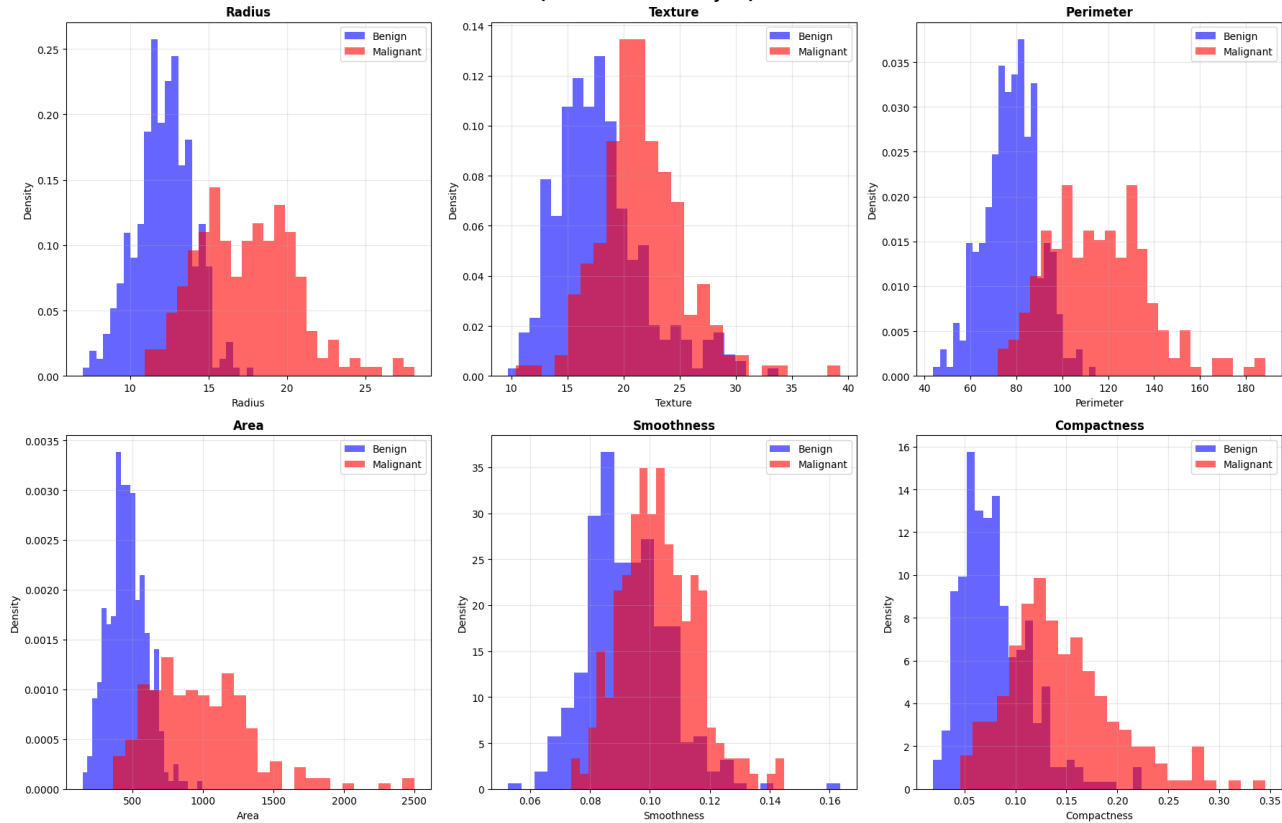
=====

DATA VISUALIZATION

=====



Feature Distributions by Diagnosis (Decision Tree Analysis)



PREPARING DATA FOR DECISION TREE

✓ Converted diagnosis: M=1 (Malignant), B=0 (Benign)
 Features (X) shape: (569, 30)
 Target (y) shape: (569,)
 Class distribution: Benign=357, Malignant=212

Applying Min-Max normalization...
 ✓ Features normalized to range [0.0, 1.0]

Dataset split (70-30):
 • Training: 398 samples
 • Testing: 171 samples
 • Training class distribution: Benign=250, Malignant=148
 • Testing class distribution: Benign=107, Malignant=64

DECISION TREE HYPERPARAMETER ANALYSIS

Testing different tree depths...

Depth	Train Acc	Test Acc	Difference	Status
1	0.9296	0.8947	0.0349	Good
2	0.9497	0.9064	0.0433	Good
3	0.9623	0.9181	0.0442	Good
4	0.9774	0.9064	0.0710	Slight Overfit
5	0.9849	0.9064	0.0785	Slight Overfit
6	0.9925	0.9123	0.0802	Slight Overfit
7	0.9975	0.8947	0.1028	Overfitting
8	1.0000	0.9006	0.0994	Slight Overfit
9	1.0000	0.9006	0.0994	Slight Overfit
10	1.0000	0.9006	0.0994	Slight Overfit
11	1.0000	0.9006	0.0994	Slight Overfit
12	1.0000	0.9006	0.0994	Slight Overfit
13	1.0000	0.9006	0.0994	Slight Overfit
14	1.0000	0.9006	0.0994	Slight Overfit
15	1.0000	0.9006	0.0994	Slight Overfit
16	1.0000	0.9006	0.0994	Slight Overfit
17	1.0000	0.9006	0.0994	Slight Overfit
18	1.0000	0.9006	0.0994	Slight Overfit
19	1.0000	0.9006	0.0994	Slight Overfit
20	1.0000	0.9006	0.0994	Slight Overfit

- ✓ Optimal max_depth: 3
- ✓ Best test accuracy: 0.9181 (91.8%)



GRID SEARCH FOR OPTIMAL PARAMETERS

Performing Grid Search...

- ✓ Best parameters found:
- criterion: gini
- max_depth: 3
- min_samples_leaf: 4
- min_samples_split: 10
- ✓ Best cross-validation score: 0.9396

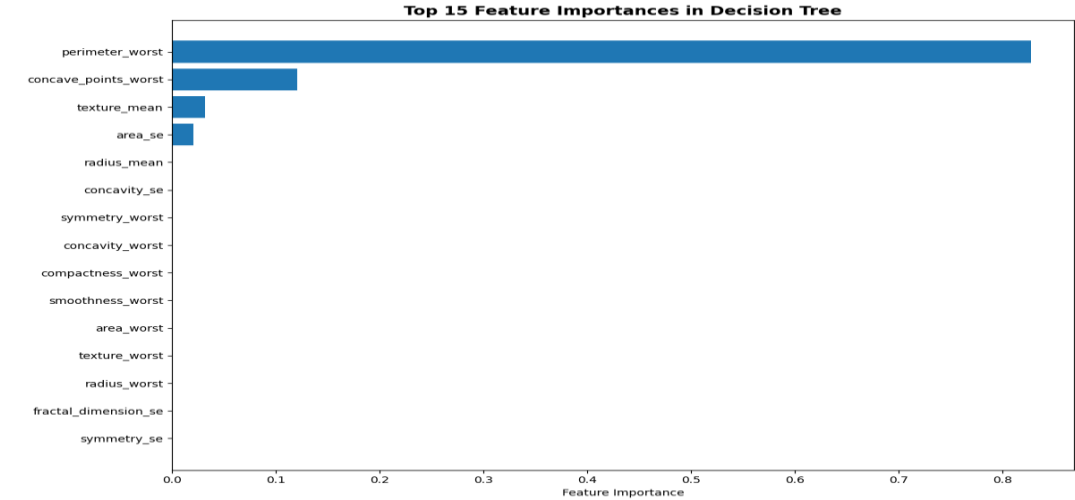
FINAL DECISION TREE MODEL

Final Model Performance:

- Training Accuracy: 0.9623 (96.2%)
- Testing Accuracy: 0.9181 (91.8%)
- Overfitting Check: 0.0442

FEATURE IMPORTANCE ANALYSIS

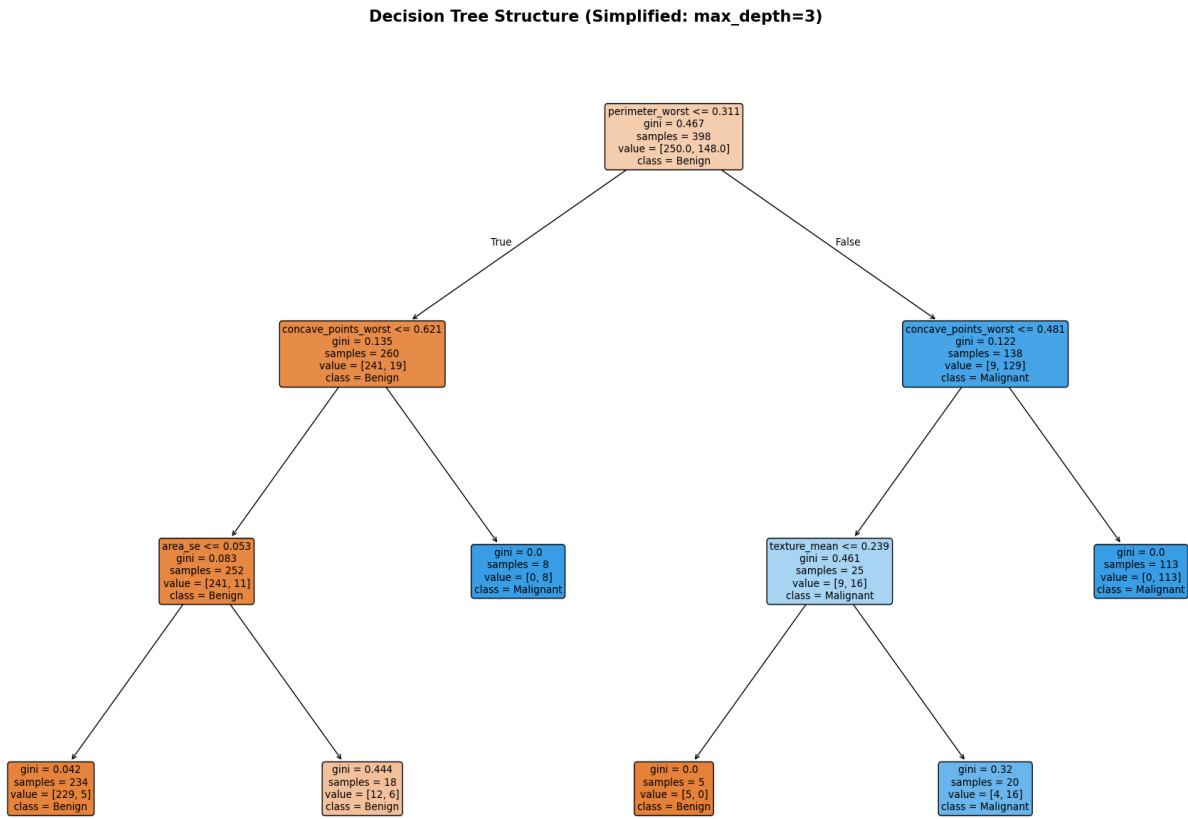
Top 10 Most Important feature		Features: importance
perimeter_worst	0.827734	
concave_points_worst	0.120496	
texture_mean	0.031655	
area_se	0.020114	
radius_mean	0.000000	
concavity_se	0.000000	
symmetry_worst	0.000000	
concavity_worst	0.000000	
compactness_worst	0.000000	
smoothness_worst	0.000000	



=====

DECISION TREE VISUALIZATION

=====



Decision Tree Rules (Top 3 levels):

```
|--- perimeter_worst <= 0.31
| |--- concave_points_worst <= 0.62
| | |--- area_se <= 0.05
| | | |--- class: 0
| | |--- area_se > 0.05
| | | |--- class: 0
| |--- concave_points_worst > 0.62
| | |--- class: 1
|--- perimeter_worst > 0.31
| |--- concave_points_worst <= 0.48
| | |--- texture_mean <= 0.24
| | | |--- class: 0
| | |--- texture_mean > 0.24
| | | |--- class: 1
| |--- concave_points_worst > 0.48
| | |--- class: 1
```

=====

COMPREHENSIVE MODEL EVALUATION

=====

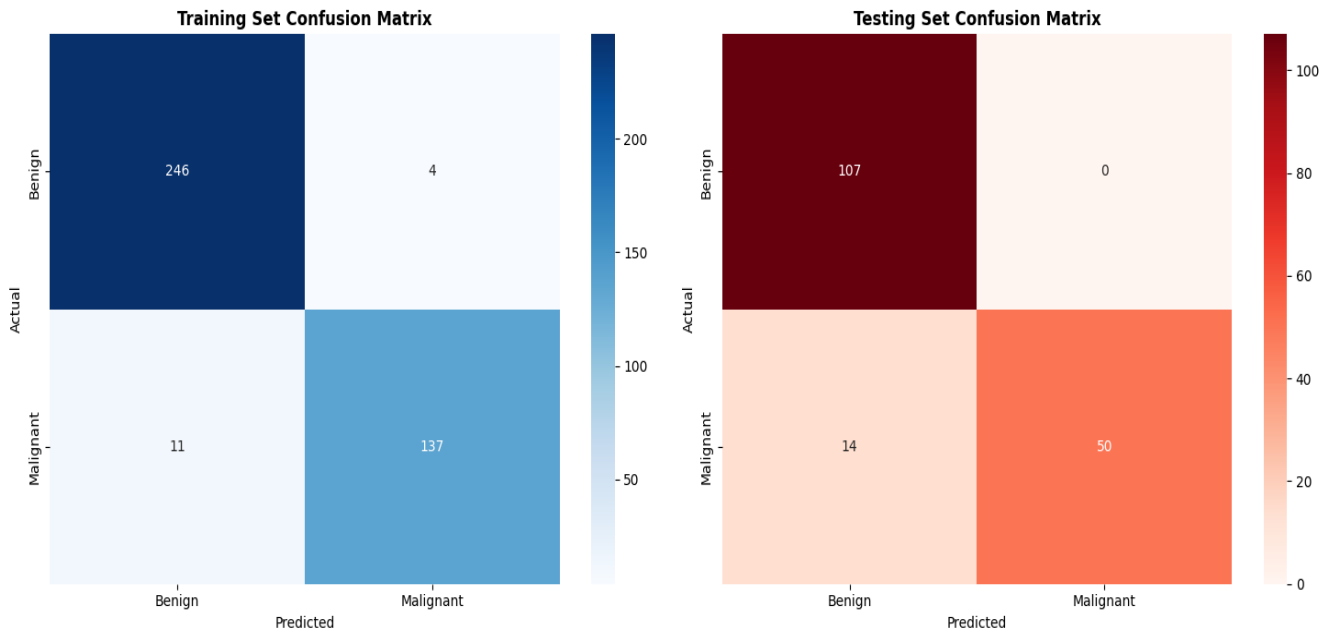
Training Set Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Benign	0.96	0.98	0.97	250
Malignant	0.97	0.93	0.95	148
accuracy			0.96	398
macro avg	0.96	0.95	0.96	398
weighted avg	0.96	0.96	0.96	398

Testing Set Classification Report:

	precision	recall	f1-score	support
Benign	0.88	1.00	0.94	107
Malignant	1.00	0.78	0.88	64
accuracy			0.92	171
macro avg	0.94	0.89	0.91	171
weighted avg	0.93	0.92	0.92	171



- Detailed Performance Metrics (Test Set):
- True Positives (Malignant correctly identified): 50
 - True Negatives (Benign correctly identified): 107
 - False Positives (Benign misclassified as Malignant): 0
 - False Negatives (Malignant misclassified as Benign): 14
 - Sensitivity (Recall for Malignant): 0.7812
 - Specificity (Recall for Benign): 1.0000
 - Precision (Malignant): 1.0000
 - F1-Score: 0.8772

- 5-Fold Cross-Validation Results:
- CV Scores: ['0.9123', '0.8947', '0.9386', '0.9474', '0.9115']
 - Mean CV Score: 0.9209 (±0.0386)

=====

EXPERIMENT SUMMARY

=====

- ✓ Dataset: 569 samples with 30 features
- ✓ Algorithm: Decision Tree Classifier
- ✓ Best Parameters: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 4, 'min_samples_split': 10}
- ✓ Final Test Accuracy: 0.9181 (91.8%)
- ✓ Cross-Validation Score: 0.9209
- ✓ Most Important Feature: perimeter_worst (0.8277342242133024)
- ✓ Model Complexity: Tree depth = 3, Leaves = 6

=====

MEDICAL INTERPRETATION

=====

- Decision Tree Advantages for Medical Diagnosis:
- Interpretable: Can trace decision path for each prediction
 - No black box: Rules are transparent and explainable
 - Feature importance: Identifies most critical diagnostic features
 - Fast prediction: Efficient for real-time diagnosis
- ⚠ Consider improving sensitivity for medical applications
- ✓ Excellent specificity: Rarely misclassifies benign cases

- Top 3 Decision Factors:
1. Perimeter Worst: 0.8277 importance

2. Concave Points Worst: 0.1205 importance
3. Texture Mean: 0.0317 importance

EXPERIMENT 7

AIM: To apply a Random Forest Classifier for classifying breast cancer tumors as benign or malignant based on features like radius and texture.

Theory:

A Random Forest is a powerful ensemble machine learning technique used for classification and regression tasks. It builds multiple decision trees during training and merges their results to improve accuracy and reduce overfitting. Each tree is trained on a random subset of the dataset, and the final prediction is made by aggregating the individual tree outputs, often using majority voting for classification. This method increases model robustness by reducing variance and bias, making it well-suited for complex problems with large datasets, such as classifying breast cancer tumors as benign or malignant. Random Forests are also known for their ability to handle missing data and identify important features in the dataset.

```
# EXPERIMENT: 7
# AIM: To apply a Random Forest Classifier for classifying breast cancer tumors # as benign or malignant
based on features like radius and texture.

# First import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier # Added missing import from
sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, classification_report

def auto_detect_structure(filename='wdbc.data'):
    """Auto-detect dataset structure and generate column names"""
    temp_data = pd.read_csv(filename, header=None)

    # Generate column names for WDBC format
    column_names = ['id', 'diagnosis']
    base_features = ['radius', 'texture', 'perimeter', 'area', 'smoothness',
                     'compactness', 'concavity', 'concave_points', 'symmetry', 'fractal_dimension']
    statistics = ['mean', 'se', 'worst']

    for stat in statistics:
        for feature in base_features:
            column_names.append(f'{feature}_{stat}')

    return column_names

# Load data with auto-detected structure
column_names = auto_detect_structure('wdbc.data')
my_data = pd.read_csv('wdbc.data', header=None, names=column_names)

print("Dataset loaded successfully!") print(f"Shape: {my_data.shape}")
my_data.head()

my_data.info()

# The line below should display (569, 32)
my_data.shape

# To remove the null values from the dataset
my_data.dropna(axis=1, inplace=True)
```

```

# Remove the id column
my_dt = my_data.drop(['id'], axis=1)
my_dt.head(3)

# Separate data by diagnosis
The_M = my_dt[my_dt.diagnosis == "M"]
The_M.head()

The_B = my_dt[my_dt.diagnosis == "B"]
The_B.head(6)

# Plot both diagnosis Benign and Malignant
plt.figure(figsize=(10, 6))
plt.title("Random Forest: Benign Tumor VS Malignant")
plt.xlabel("Radius_Mean")
plt.ylabel("Texture_Mean")
plt.scatter(The_M.radius_mean, The_M.texture_mean, color="red", label="Malignant", alpha=0.6)
plt.scatter(The_B.radius_mean, The_B.texture_mean, color="blue", label="Benign", alpha=0.6)
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig("random_forest_plot.png", facecolor='w', bbox_inches='tight', pad_inches=0.3)
plt.show()

# Convert diagnosis to numerical values
my_dt.diagnosis = [1 if i == "M" else 0 for i in my_dt.diagnosis]

# Separate features and target
x_val = my_dt.drop(["diagnosis"], axis=1)
y_val = my_dt.diagnosis.values

# Min-Max Normalization
scaler = MinMaxScaler()
my_x = scaler.fit_transform(x_val)
my_x = pd.DataFrame(my_x, columns=x_val.columns)

print(f"Features shape: {my_x.shape}")
print(f"Target shape: {y_val.shape}")

# Split the dataset into training and testing sets (60% training, 40% testing)
x_train, x_test, y_train, y_test = train_test_split(my_x, y_val, test_size=0.4, random_state=41)

print(f"Training set: {x_train.shape[0]} samples")
print(f"Testing set: {x_test.shape[0]} samples")

# Now let's implement the Random Forest classifier
my_rf = RandomForestClassifier(n_estimators=100, random_state=42)
my_rf.fit(x_train, y_train)

print("Random Forest model trained successfully!")

# Get predictions for both training and testing sets
y_train_pred = my_rf.predict(x_train)
y_test_pred = my_rf.predict(x_test)

# The score() is used to display the classifier accuracy
train_accuracy = my_rf.score(x_train, y_train)
test_accuracy = my_rf.score(x_test, y_test)

print(f"\nRandom Forest Performance:")
print(f"Training Accuracy: {train_accuracy:.4f} ({train_accuracy*100:.1f}%)")
print(f"Testing Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.1f}%)")

# Training confusion matrix (FIXED)
print("\nTraining Confusion Matrix:")

```

```

cm_train = confusion_matrix(y_train, y_train_pred) # Fixed: actual vs predicted
print(cm_train)

# Training classification report (FIXED)
print("\nClassification Report - Training:")
print(classification_report(y_train, y_train_pred, target_names=['Benign', 'Malignant']))

# Testing confusion matrix (FIXED)
print("\nTesting Confusion Matrix:")
cm_test = confusion_matrix(y_test, y_test_pred) # Fixed: actual vs predicted
print(cm_test)

# Testing classification report (FIXED)
print("\nClassification Report - Testing:")
print(classification_report(y_test, y_test_pred, target_names=['Benign', 'Malignant']))

# Feature importance (bonus - simple addition)
print(f"\nTop 10 Most Important Features:")
feature_importance = pd.DataFrame({
    'feature': x_val.columns,
    'importance': my_rf.feature_importances_
}).sort_values('importance', ascending=False)

print(feature_importance.head(10).to_string(index=False))

# Simple visualization of results
plt.figure(figsize=(12, 5))

# Plot 1: Confusion matrix for test set
plt.subplot(1, 2, 1)
plt.imshow(cm_test, interpolation='nearest', cmap='Blues')
plt.title("Test Set Confusion Matrix")
plt.colorbar()
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.xticks([0, 1], ['Benign', 'Malignant'])
plt.yticks([0, 1], ['Benign', 'Malignant'])

# Add text annotations
for i in range(2):
    for j in range(2):
        plt.text(j, i, cm_test[i, j], ha='center', va='center', fontsize=14, fontweight='bold')

# Plot 2: Feature importance
plt.subplot(1, 2, 2)
top_features = feature_importance.head(10)
plt.barh(range(len(top_features)), top_features['importance'])
plt.yticks(range(len(top_features)), top_features['feature'])
plt.xlabel('Importance')
plt.title("Top 10 Feature Importances")
plt.gca().invert_yaxis()

plt.tight_layout()
plt.savefig("random_forest_results.png", dpi=300, bbox_inches='tight')
plt.show()

# Summary
print(f"\n{'='*50}")
print("EXPERIMENT SUMMARY")
print(f"\n{'='*50}")
print(f"✓ Dataset: {len(my_dt)} samples with {x_val.shape[1]} features")
print(f"✓ Algorithm: Random Forest with {my_rf.n_estimators} trees")
print(f"✓ Training Accuracy: {train_accuracy:.4f}")
print(f"✓ Testing Accuracy: {test_accuracy:.4f}")
print(f"✓ Most Important Feature: {feature_importance.iloc[0]['feature']}")

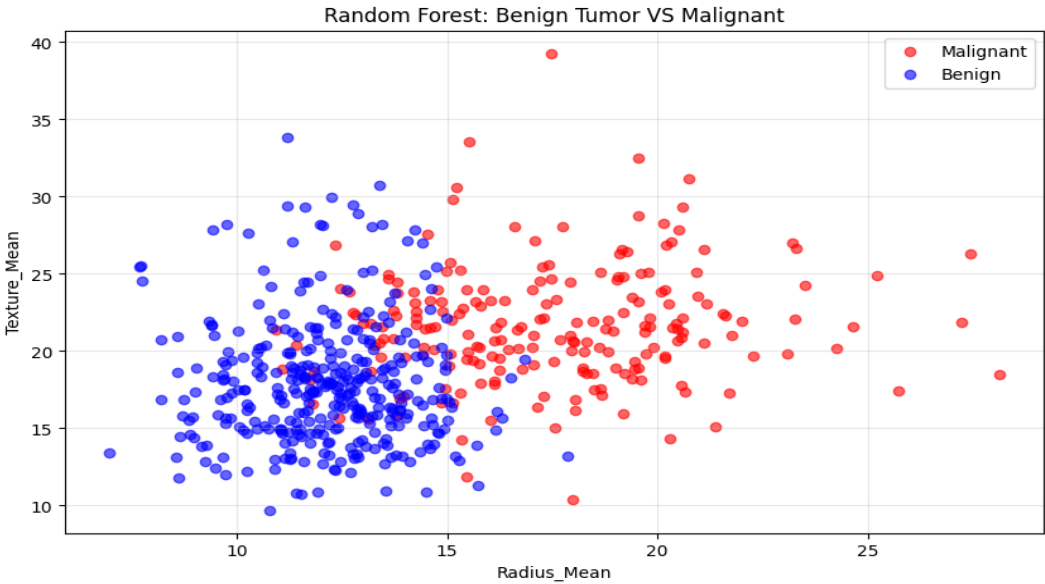
```

```
if test_accuracy > 0.95:
    print("✓ Excellent performance achieved!")
elif test_accuracy > 0.90:
    print("✓ Good performance achieved!")
else:
    print("⚠ Consider tuning parameters for better performance")
```

Dataset loaded successfully!
Shape: (569, 32)
<class 'pandas.core.frame.DataFrame'> RangeIndex:
569 entries, 0 to 568
Data columns (total 32 columns):

#	Column	Non-Null Count		Dtype
0	id	569	non-null	int64
1	diagnosis	569	non-null	object
2	radius_mean	569	non-null	float64
3	texture_mean	569	non-null	float64
4	perimeter_mean	569	non-null	float64
5	area_mean	569	non-null	float64
6	smoothness_mean	569	non-null	float64
7	compactness_mean	569	non-null	float64
8	concavity_mean	569	non-null	float64
9	concave_points_mean	569	non-null	float64
10	symmetry_mean	569	non-null	float64
11	fractal_dimension_mean	569	non-null	float64
12	radius_se	569	non-null	float64
13	texture_se	569	non-null	float64
14	perimeter_se	569	non-null	float64
15	area_se	569	non-null	float64
16	smoothness_se	569	non-null	float64
17	compactness_se	569	non-null	float64
18	concavity_se	569	non-null	float64
19	concave_points_se	569	non-null	float64
20	symmetry_se	569	non-null	float64
21	fractal_dimension_se	569	non-null	float64
22	radius_worst	569	non-null	float64
23	texture_worst	569	non-null	float64
24	perimeter_worst	569	non-null	float64
25	area_worst	569	non-null	float64
26	smoothness_worst	569	non-null	float64
27	compactness_worst	569	non-null	float64
28	concavity_worst	569	non-null	float64
29	concave_points_worst	569	non-null	float64
30	symmetry_worst	569	non-null	float64
31	fractal_dimension_worst	569	non-null	float64

dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB



Features shape: (569, 30)
Target shape: (569,)
Training set: 341 samples
Testing set: 228 samples
Random Forest model trained successfully!

Random Forest Performance:
Training Accuracy: 1.0000 (100.0%)
Testing Accuracy: 0.9518 (95.2%)

Training Confusion Matrix:
[[221 0]
[0 120]]

Classification Report - Training:

	precision	recall	f1-score	support
Benign	1.00	1.00	1.00	221
Malignant	1.00	1.00	1.00	120
accuracy			1.00	341
macro avg	1.00	1.00	1.00	341
weighted avg	1.00	1.00	1.00	341

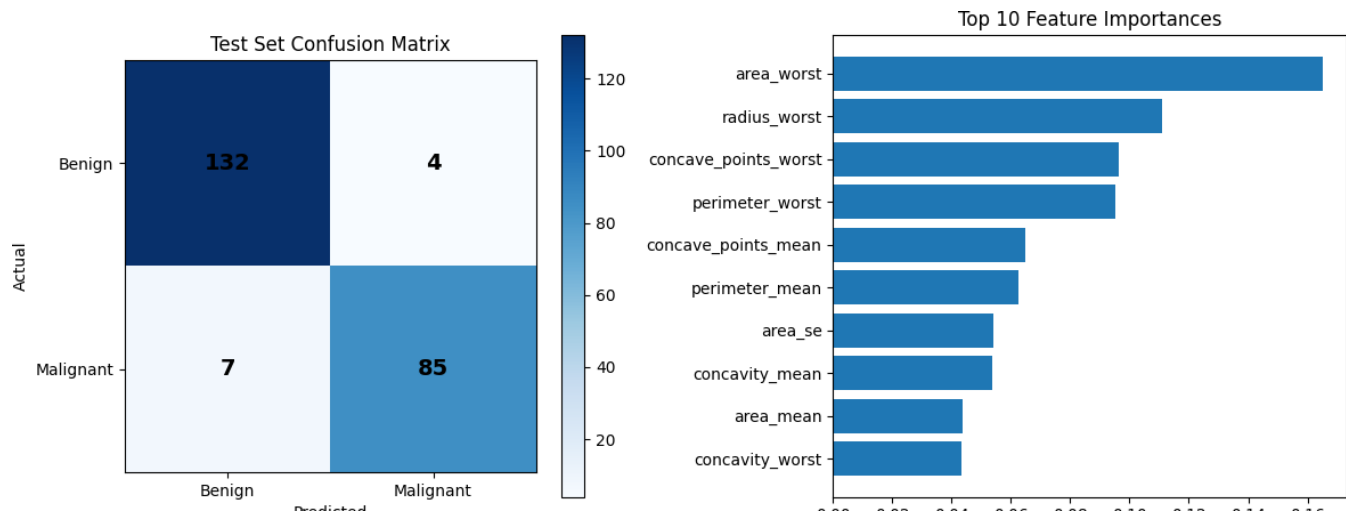
Testing Confusion Matrix:
[[132 4]
[7 85]]

Classification Report - Testing:

	precision	recall	f1-score	support
Benign	0.95	0.97	0.96	136
Malignant	0.96	0.92	0.94	92
accuracy			0.95	228
macro avg	0.95	0.95	0.95	228
weighted avg	0.95	0.95	0.95	228

Top 10 Most Important Features:

feature	importance
area_worst	0.165004
radius_worst	0.111081
concave_points_worst	0.096306
perimeter_worst	0.095376
concave_points_mean	0.064765
perimeter_mean	0.062571
area_se	0.054028
concavity_mean	0.053555
area_mean	0.043559
concavity_worst	0.043303



predicted

0.00 0.02 0.04 0.06 0.08 0.10 0.12 0.14 0.16
Importance

=====

EXPERIMENT SUMMARY

=====

- ✓ Dataset: 569 samples with 30 features
- ✓ Algorithm: Random Forest with 100 trees
- ✓ Training Accuracy: 1.0000
- ✓ Testing Accuracy: 0.9518
- ✓ Most Important Feature: area_worst
- ✓ Excellent performance achieved!

EXPERIMENT 8

AIM: Use Support Vector Machine to perform the classification and regression on given dataset.

Objectives:

- Import the dataset and perform the classification using SVM
- Evaluate the model's performance on testing dataset and validation dataset
- Use Sklearn's Grid Search CV method for fine tuning of hyper-parameters

Theory:

Support Vector Machine (SVM) is a supervised machine learning algorithm widely used for classification and regression tasks. The primary goal of SVM is to find a hyper-plane that best separates data points of different classes. This separation is often challenging in cases where data is not linearly separable, which is where kernel functions become valuable. A kernel function maps data into a higher-dimensional space, allowing SVM to draw complex decision boundaries.

In this code, we experiment with four types of kernels: polynomial, radial basis function (RBF), sigmoid, and linear. The polynomial kernel maps data to a polynomial feature space, effectively handling complex patterns. The RBF kernel, a Gaussian-like similarity measure, is effective in capturing intricate non-linear relationships, especially in high-dimensional spaces. The sigmoid kernel mimics neural network activation functions and can be useful for specific types of non-linear data. The linear kernel, on the other hand, is ideal for linearly separable data and draws a straight boundary between classes.

To further enhance the model, we use GridSearchCV for hyperparameter optimization. GridSearchCV tests different values of 'C' (the penalty parameter) and 'gamma' (which influences the impact of each data point) across all kernels to find the best-performing combination, thereby maximizing classification accuracy. This optimization is crucial in identifying the most effective SVM configuration for the iris dataset classification problem.

```
# Imports
import pandas as pd
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Loading iris dataset
print("Loading Iris Dataset...")
iris = datasets.load_iris()

# Create DataFrames for better visualization
iris_data_df = pd.DataFrame(iris.data, columns=iris.feature_names)
iris_target_df = pd.DataFrame(iris.target, columns=['class'])

print("Dataset Info:")
print(f'Dataset shape: {iris_data_df.shape}')
print(f'Target classes: {iris.target_names}') #['setosa' 'versicolor' 'virginica']
print("\nFirst 5 rows of features:")
print(iris_data_df.head())
print("\nFirst 5 rows of targets:")
print(iris_target_df.head())
```

```

# Prepare data
X = iris.data # Features
y = iris.target # Target

# Split data into train, validation, and test sets
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp)

print(f"\nData split:")
print(f"Training set: {X_train.shape[0]} samples")
print(f"Validation set: {X_val.shape[0]} samples")
print(f"Test set: {X_test.shape[0]} samples")

# Function to evaluate model
def evaluate_model(model, X_test, y_test, kernel_name):
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"\n=== {kernel_name} Kernel Results ===")
    print(f"Accuracy: {accuracy:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, target_names=iris.target_names))
    return y_pred, accuracy

# Test different kernels
kernels_results = {}

# 1. POLYNOMIAL KERNEL (reduced degree to avoid overfitting)
print("\n" + "="*50)
svc_poly = SVC(kernel='poly', degree=3, gamma='scale', random_state=42)

# Changed degree from 8 to 3
svc_poly.fit(X_train, y_train)
y_pred_poly, acc_poly = evaluate_model(svc_poly, X_val, y_val, "Polynomial")
kernels_results['Polynomial'] = acc_poly

# 2. RADIAL BASIS FUNCTION (RBF) KERNEL
svc_rbf = SVC(kernel='rbf', gamma='scale', random_state=42)

# Changed from 'auto' to 'scale'
svc_rbf.fit(X_train, y_train)
y_pred_rbf, acc_rbf = evaluate_model(svc_rbf, X_val, y_val, "RBF")
kernels_results['RBF'] = acc_rbf

# 3. SIGMOID KERNEL
svc_sigmoid = SVC(kernel='sigmoid', gamma='scale', random_state=42)
svc_sigmoid.fit(X_train, y_train)
y_pred_sigmoid, acc_sigmoid = evaluate_model(svc_sigmoid, X_val, y_val, "Sigmoid")
kernels_results['Sigmoid'] = acc_sigmoid

# 4. LINEAR KERNEL
svc_linear = SVC(kernel='linear', random_state=42)

# Removed gamma (not used for linear)
svc_linear.fit(X_train, y_train)
y_pred_linear, acc_linear = evaluate_model(svc_linear, X_val, y_val, "Linear")
kernels_results['Linear'] = acc_linear

# Compare kernel performances
print("\n" + "="*50)
print("KERNEL COMPARISON ON VALIDATION SET:")
print("="*50)
for kernel, accuracy in kernels_results.items():
    print(f"{kernel:12}: {accuracy:.4f}")

# Hyperparameter tuning using GridSearchCV
print(f"\n{'='*50}")

```

```

print("HYPERPARAMETER TUNING WITH GRIDSEARCHCV")
print("="*50)

# Define parameter grid (removed gamma for linear kernel)
param_grid = [
    # For RBF, polynomial, and sigmoid kernels
    {'kernel': ['rbf'], 'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto', 0.1, 0.01, 0.001]},
    {'kernel': ['poly'], 'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto', 0.1, 0.01, 0.001], 'degree': [2, 3, 4]},
    {'kernel': ['sigmoid'], 'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto', 0.1, 0.01, 0.001]},
    # For linear kernel (no gamma parameter)
    {'kernel': ['linear'], 'C': [0.1, 1, 10, 100]}
]

# Perform GridSearchCV
grid_search = GridSearchCV(
    SVC(random_state=42),
    param_grid,
    cv=5, # 5-fold cross-validation
    scoring='accuracy',
    verbose=1,
    n_jobs=-1 # Use all available cores
)

print("Performing Grid Search...")
grid_search.fit(X_train, y_train)

print(f"\nBest parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")

# Evaluate best model on validation set
best_model = grid_search.best_estimator_
y_val_pred = best_model.predict(X_val)
val_accuracy = accuracy_score(y_val, y_val_pred)
print(f"Validation accuracy with best model: {val_accuracy:.4f}")

# Final evaluation on test set
y_test_pred = best_model.predict(X_test)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f"\n{'='*50}")
print("FINAL TEST SET EVALUATION")
print("="*50)
print(f"Test accuracy: {test_accuracy:.4f}")
print("\nFinal Classification Report:")
print(classification_report(y_test, y_test_pred, target_names=iris.target_names))

# Visualizations
plt.figure(figsize=(15, 5))

# 1. Kernel comparison
plt.subplot(1, 3, 1)
kernels = list(kernels_results.keys())
accuracies = list(kernels_results.values())
bars = plt.bar(kernels, accuracies, color=['skyblue', 'lightgreen', 'salmon', 'gold'])
plt.title('Kernel Performance Comparison\n(Validation Set)')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{acc:.3f}', ha='center', va='bottom')
plt.xticks(rotation=45)

# 2. Confusion Matrix for best model
plt.subplot(1, 3, 2)

```

```

cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
plt.title('Confusion Matrix\n(Best Model on Test Set)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')

# 3. Feature importance visualization (for linear kernel if it's the best)
plt.subplot(1, 3, 3)

if hasattr(best_model, 'coef_') and best_model.kernel == 'linear':
    feature_importance = np.abs(best_model.coef_[0])
    features = iris.feature_names
    bars = plt.bar(range(len(features)), feature_importance, color='lightcoral')
    plt.title('Feature Importance\n(Linear SVM)')
    plt.xlabel('Features')
    plt.ylabel('|Coefficient|')
    plt.xticks(range(len(features)), features, rotation=45, ha='right')
else:
    # If not linear, show a simple accuracy comparison
    models = ['Best Model', 'Random Baseline']
    scores = [test_accuracy, 1/3] # Random baseline for 3-class problem
    bars = plt.bar(models, scores, color=['green', 'red'])
    plt.title('Model Performance')
    plt.ylabel('Accuracy')
    plt.ylim(0, 1)
    for bar, score in zip(bars, scores):
        plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                f'{score:.3f}', ha='center', va='bottom')

plt.tight_layout()
plt.show()

print(f"\n{'='*50}")
print("EXPERIMENT COMPLETED SUCCESSFULLY!")
print(f"{'='*50}")

```



Loading Iris Dataset...

Dataset Info:

Dataset shape: (150, 4)

Target classes: ['setosa' 'versicolor' 'virginica']

First 5 rows of features:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	
0	5.1		3.5	1.4	0.2
1	4.9		3.0	1.4	0.2
2	4.7		3.2	1.3	0.2
3	4.6		3.1	1.5	0.2
4	5.0		3.6	1.4	0.2

First 5 rows of targets: class

0	0
1	0
2	0
3	0
4	0

Data split:

Training set: 90 samples

Validation set: 30 samples Test set: 30 samples

=====

=== Polynomial Kernel Results === Accuracy:
0.9333

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

=== RBF Kernel Results === Accuracy:
0.9333

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

=== Sigmoid Kernel Results === Accuracy:
0.1333

Classification Report:

	precision	recall	f1-score	support
setosa	0.18	0.40	0.25	10
versicolor	0.00	0.00	0.00	10
virginica	0.00	0.00	0.00	10
accuracy			0.13	30
macro avg	0.06	0.13	0.08	30
weighted avg	0.06	0.13	0.08	30

